Attention-Logit Hijacking in GPT-2

How a small set of late-layer attention heads raise the raw compatibility score (logit) for an incorrect, merely co-occurring entity above every competing candidate, so softmax amplifies that edge into a dominant next-token probability -- causing GPT-2 to ignore negation entirely.

Headline finding: across every negation prompt tested, the wrong answer wins not because of a softmax quirk, but because its raw pre-softmax attention LOGIT (the scaled Q·K compatibility score) genuinely rises above every competing candidate's logit, specifically in layers 8-11. A single head, **Layer 10 / Head 7**, shows this crossover in all four negation cases tested, with its raw logit for the wrong entity climbing from roughly -7 at layer 3 to +1 to +4 by layer 9-11, while the correct answer's logit never leaves negative territory. Softmax then takes this modest logit edge and turns it into 83-92% next-token probability for the wrong answer. This report documents that evidence directly, situates it against recent (2026) mechanistic-interpretability literature on negation in transformers, and logs every supporting plot in the appendix.

Model	GPT-2 124M (original OpenAI checkpoint)
Test cases	5 prompts (test_prompts.txt), temperature=1, top_k=40, seed=0, length=20
Candidates compared	Paris, Chicago, Berlin, London, Rome
True / hijack labels	true_answer=Paris; hijack_word=Chicago (cases 1-2); auto-detected for 3-4
Analysis script	src/attention_scores.py (model.py modified to expose pre-softmax attention logits)

Methodology

For each prompt, GPT-2 generates a continuation, then the full sequence is fed back through the model in one forward pass to recover exact values at three distinct points in the computation graph:

1. Attention logits -- the raw, scaled Q·K score for one head/layer, before softmax (model.py: `w = w * rsqrt(d); w = mask(w)`, captured here). Unbounded, can be negative, and is NOT normalized against competing keys.
2. Attention weights -- the same tensor after softmax. Bounded [0,1], sums to 1 across all keys per head -- this is what the model actually uses to mix Value vectors.
3. Output-layer probability -- softmax(ln_f(h) @ wte^T), the model's actual next-token prediction, entirely downstream of all 12 attention layers.

Named-entity tokens are located via BPE re-encoding (since e.g. "Eiffel" splits into 3 sub-word tokens) and tracked at the query position -- the last prompt token, whose output determines the first generated word. Full per-layer seq×seq heatmaps use one representative head (0) rather than a head-average, since averaging smears out the sharp per-head patterns; causally-masked cells are shown in flat gray and each panel is scaled to its own range using a diverging RdBu_r palette (logits centered at zero, weights centered at each panel's own mean), styled after Figure 2 of Zhang et al. 2023 ("Efficient Streaming Language Models with Attention Sinks"). True-answer and hijacking-token occurrences are marked green/red.

Quantitative Evidence

Candidate probability by case

P(candidate | context), computed via teacher-forced softmax at the query position -- the model's actual next-token prediction, not sampled or approximated. Correct answer highlighted green.

Case 0 (baseline, no negation)
The Eiffel Tower is in the city of

Candidate	P(candidate | context)
Paris	0.0700 <- correct
London	0.0584
Berlin	0.0235
Chicago	0.0067
Rome	0.0062

Case 1 (single negation)
The Eiffel Tower is not in Chicago. Therefore, the Eiffel Tower is in ...

Candidate	P(candidate | context)
Chicago	0.8307
London	0.0027
Paris	0.0023 <- correct
Berlin	0.0005
Rome	0.0002

Case 2 (negation repeated 6x)
(6x) The Eiffel Tower is not in Chicago. ... Therefore, the Eiffel Tow...

Candidate	P(candidate | context)
Chicago	0.9210
Paris	0.0008 <- correct
London	0.0006
Berlin	0.0002
Rome	0.0001

Case 3 (entities swapped: Paris negated)
The Eiffel Tower is not in Paris. Therefore, the Eiffel Tower is in th...

Candidate	P(candidate | context)
Paris	0.4466 <- correct
London	0.0160
Rome	0.0093
Berlin	0.0083
Chicago	0.0016

Case 4 (different template, France/Berlin)
The capital of France is not Berlin. Therefore, the capital of France ...

Candidate	P(candidate | context)
Paris	0.1293 <- correct
Berlin	0.0857
London	0.0206
Rome	0.0086
Chicago	0.0004

The logit crossover (Case 1, Chicago@position 8)

The clearest mechanistic evidence: tracking the raw attention logit (averaged over heads) from the query position to the hijacking token "Chicago" vs. every other keyword token, layer by layer. Chicago is the ONLY one whose logit crosses from negative into positive territory -- everything else stays negative through all 12 layers.

Layer	Chicago logit (avg over heads)	Other keyword tokens (Eiffel/Tower)
3	-7.2	-8.0 to -11.0
7	-1.6	-5.0 to -6.6
8	-1.4	-3.0 to -4.0
9	0.0 (crosses zero)	-3.2 to -3.8
10	+0.6	-2.0
11	+1.0	-0.7

Top attending heads (weight vs. logit), per case

Case 1 (Chicago@8): weight top-5: L9H9=0.586, L10H7=0.547, L10H0=0.434, L9H8=0.303, L11H10=0.281
logit top-5: L9H9=4.49, L10H7=3.30, L11H10=3.25, L9H6=3.15, L10H10=2.84

Case 2 (Chicago@8): weight top-5: L9H9=0.355, L10H7=0.278, L9H8=0.220, L10H0=0.205, L9H6=0.132
logit top-5: L9H9=4.03, L10H7=3.21, L9H6=2.96, L11H10=2.76, L7H10=2.58

Case 3 (Paris@8): weight top-5: L10H7=0.678, L9H0=0.362, L9H9=0.333, L9H8=0.312, L11H10=0.308
logit top-5: L10H7=3.97, L10H11=3.60, L9H9=3.58, L11H10=3.31, L10H10=2.76

Case 4 (Berlin@6): weight top-5: L10H7=0.542, L10H3=0.519, L9H6=0.433, L8H8=0.422, L7H7=0.357
logit top-5: L10H3=4.41, L9H6=4.11, L7H7=3.44, L10H7=3.38, L9H9=2.89

Head tally across all four negation cases

Head	Appears in cases	Weights
L10H7	1, 2, 3, 4	0.547, 0.278, 0.678, 0.542
L9H9	1, 2, 3	0.586, 0.355, 0.333
L9H8	1, 2, 3	0.303, 0.220, 0.312
L10H0	1, 2	0.434, 0.205
L11H10	1, 3	0.281, 0.308
L9H6	2, 4	0.132, 0.433
L9H0	3	0.362
L10H3	4	0.519
L8H8	4	0.422
L7H7	4	0.357

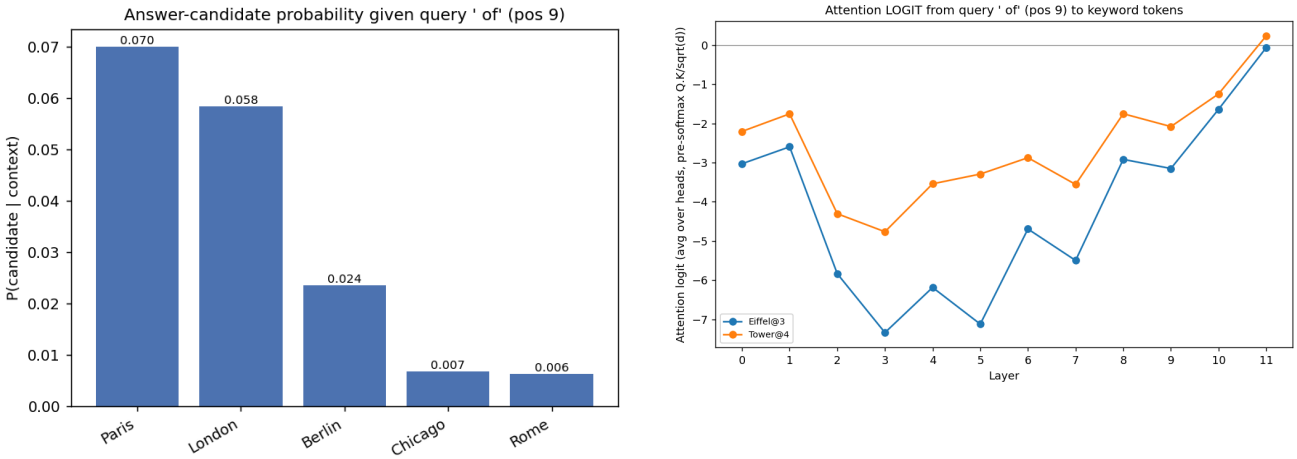
L10H7 is the single most consistent hijacking head -- present in the top-5 attending heads for every negation case regardless of which entity is negated (Chicago, repeated Chicago, Paris, or Berlin), with weight 0.28-0.68 and logit +2.76 to +4.41. It is not encoding any one city -- it is encoding "most salient recently/repeatedly named proper noun," independent of the negation preceding it.

Illustrative Plots

A hand-picked subset of the most diagnostic plots per case. The complete set of every plot generated (40 images across 5 cases) is logged in the Appendix.

Case 0: baseline, no negation

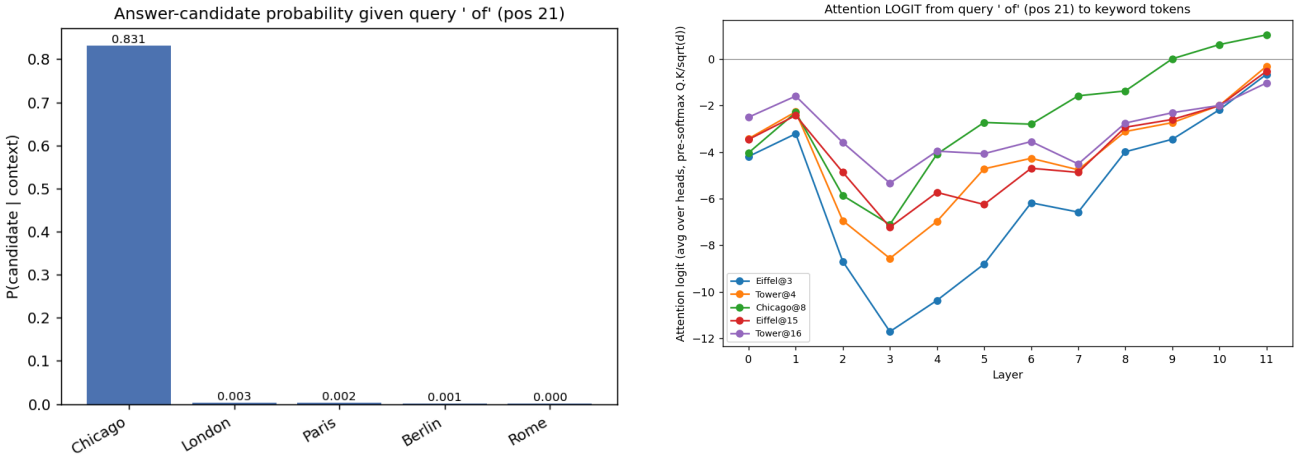
Prompt: The Eiffel Tower is in the city of **Generated:** *Paris.*



Left: output-layer P(candidate). Right: raw attention LOGIT per layer, per keyword occurrence.

Case 1: single negation

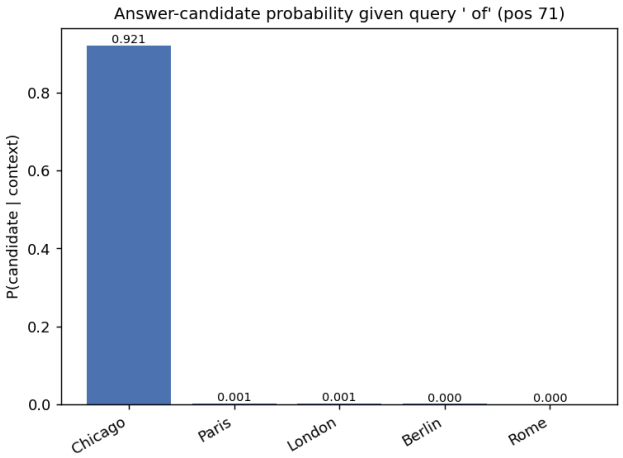
Prompt: The Eiffel Tower is not in Chicago. Therefore, the Eiffel Tower is in the city of **Generated:** *Chicago.*

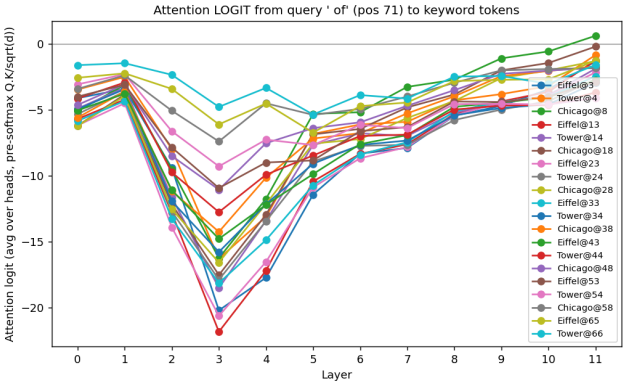


Left: output-layer P(candidate). Right: raw attention LOGIT per layer, per keyword occurrence.

Case 2: negation repeated 6x

Prompt: (6x) The Eiffel Tower is not in Chicago. ... Therefore, the Eiffel Tower is in the city of **Generated:** *Chicago.*

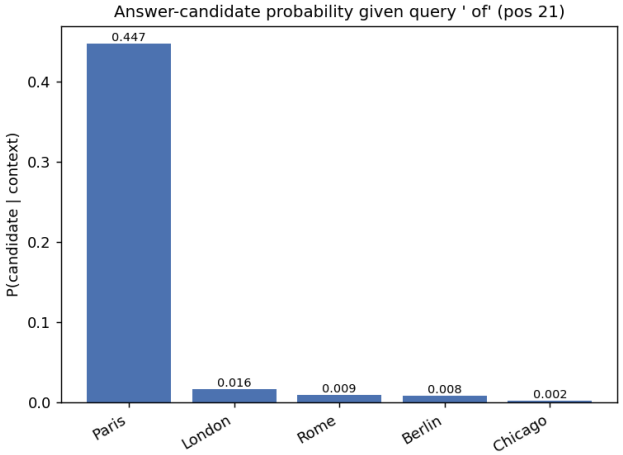


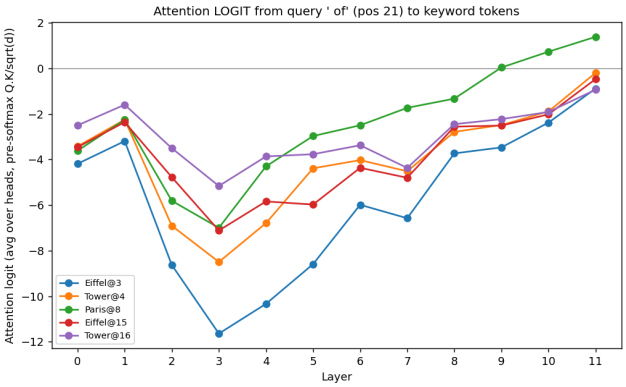


Left: output-layer P(candidate). Right: raw attention LOGIT per layer, per keyword occurrence.

Case 3: entities swapped: Paris negated

Prompt: The Eiffel Tower is not in Paris. Therefore, the Eiffel Tower is in the city of **Generated:** *New York.*

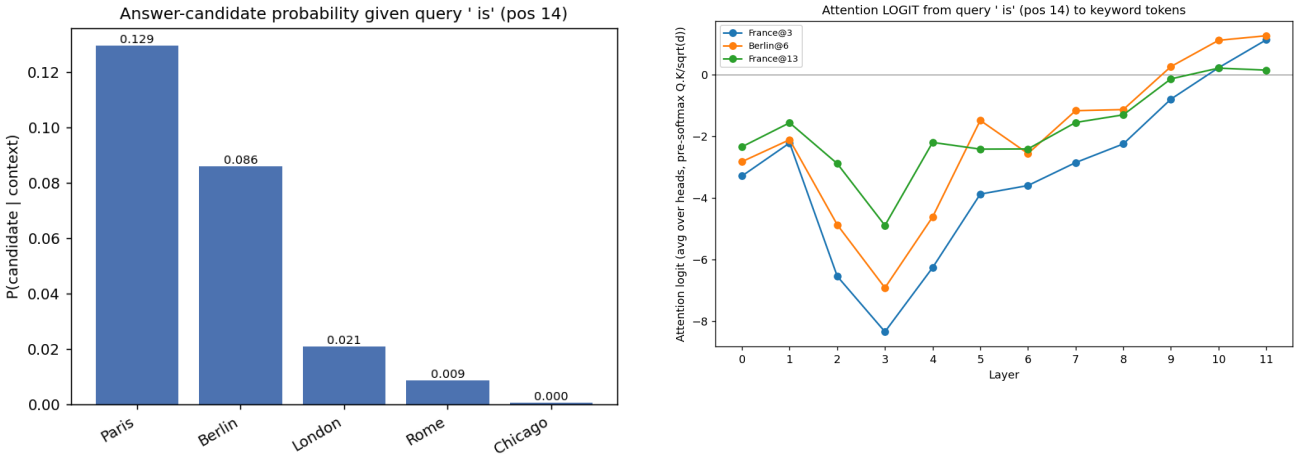




Left: output-layer P(candidate). Right: raw attention LOGIT per layer, per keyword occurrence.

Case 4: different template, France/Berlin

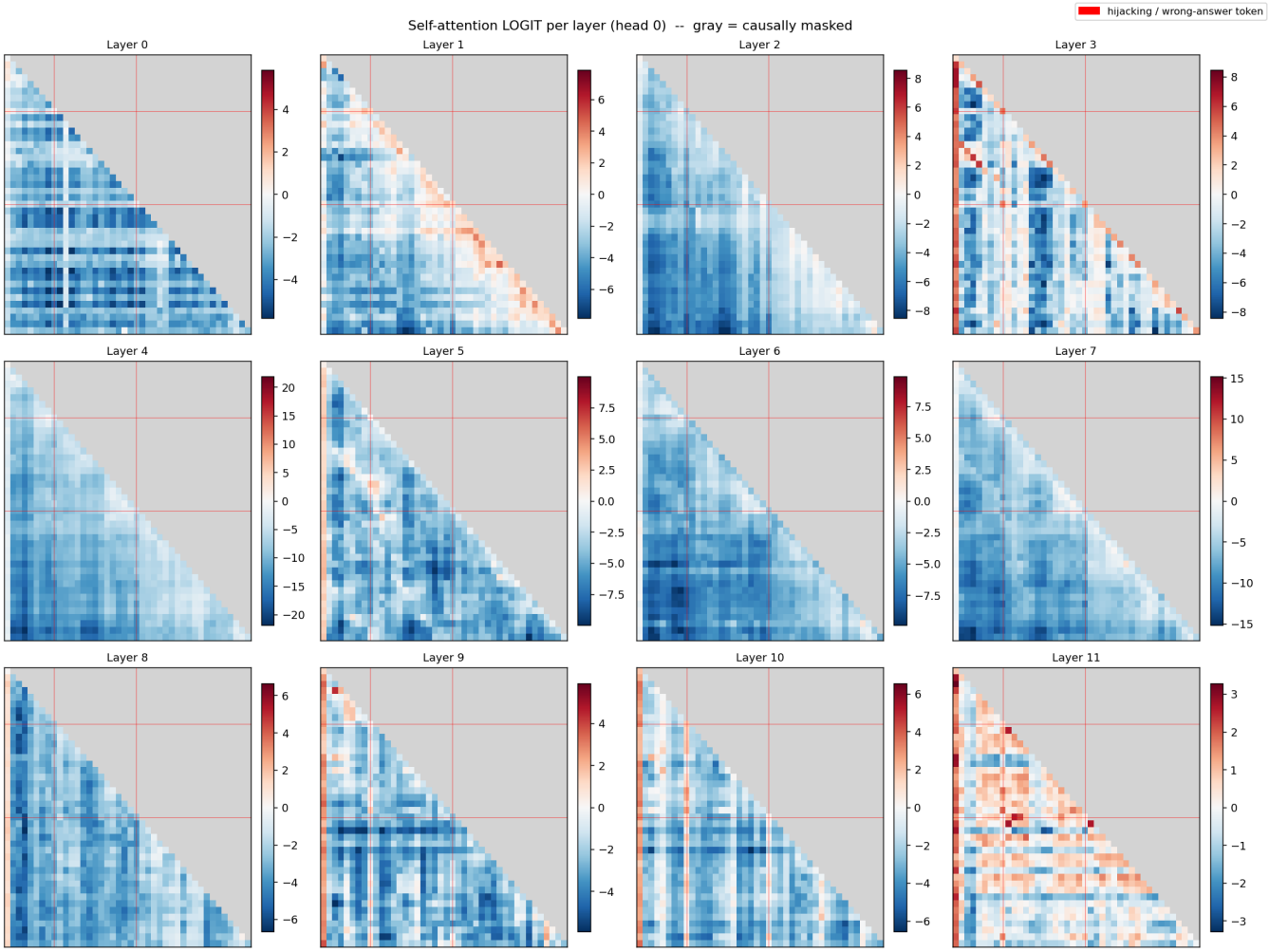
Prompt: The capital of France is not Berlin. Therefore, the capital of France is **Generated:** *not Berlin.*



Left: output-layer P(candidate). Right: raw attention LOGIT per layer, per keyword occurrence.

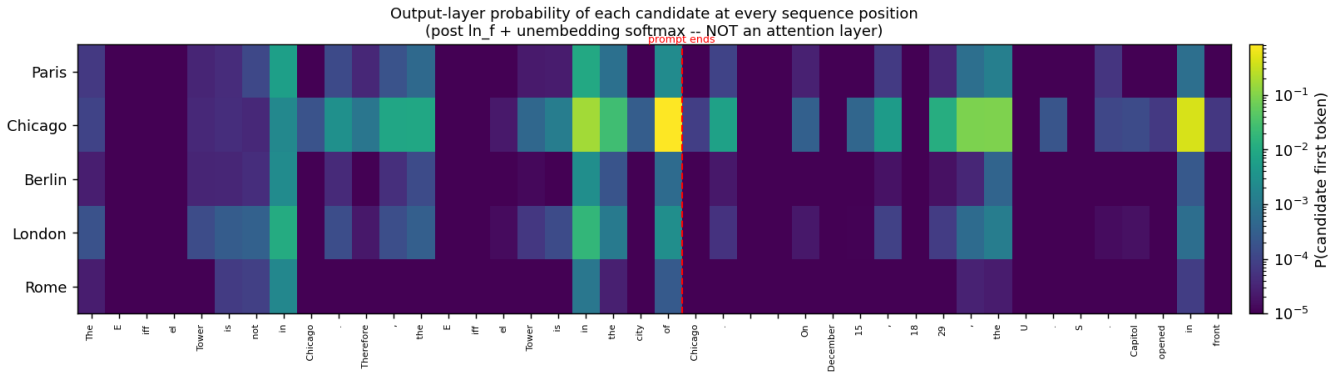
Case 1 -- full per-layer logit heatmap (the clearest single picture)

Every layer's raw attention logit matrix (head 0), causally masked in gray, diverging palette centered at zero. Red grid-lines mark every occurrence of the hijacking token "Chicago."



Case 1 -- output-layer probability across the whole sequence

Shows exactly where/when in the sequence the output layer's belief in "Chicago" spikes -- not just at the one decision point, but recurrently, every time "in" or "Chicago" appears later in the generated text.



Literature Review

A web search for recent mechanistic-interpretability work on negation in transformer language models surfaced three directly relevant results.

Interpreting Negation in GPT-2: Layer- and Head-Level Causal Analysis

Al Mofael, Kuhn, Alkadi & Yang. arXiv:2603.12423 (2026 IEEE CCWC). Studies the exact model used in this report -- GPT-2 Small -- with activation patching and head ablation on a 12,000-pair matched affirmative/negated sentence dataset, using a custom "Negation Effect Score" (NES). **Finding:** negation sensitivity is highly concentrated in a small number of **mid-layer heads, mainly layers 4-6**. Ablating them weakens negation sensitivity; restoring cached affirmative activations recovers it, read as evidence that these heads carry a genuine affirmative/negative distinction. **This is diagnostic only -- no fix is proposed.** It is consistent with, and complementary to, the present report: layers 4-6 appear to be where the negated representation gets built, well before our hijacking heads (layers 9-11) override it.

How Language Models Process Negation

Zhou, Zhou, Jia & May. arXiv:2605.03052 (ICML 2026). Studies Llama-3.1-8B, Mistral-7B, Qwen2.5/3, Gemma-2 and OLMo-2. **Finding:** these models already contain internal components that process negation correctly, but black-box accuracy is poor because of **late-layer attention modules that promote a simple "shortcut" answer** instead. Two internal mechanisms are tested -- suppression (attending to the negated phrase and inhibiting related concepts) and construction (directly building a representation of the whole negated phrase) -- both occur, with construction more prominent. **Proposed fix -- "Cumulative Attention Sink":** for the final token, force every attention head from a chosen late layer onward to attend only to itself and the first/BOS token, blocking that layer range from looking at the shortcut-triggering entity at all. Training-free, no tuning required. **Results (their Table 3):** negation accuracy rose from 50.5% to 67.8% on Llama-3.1-8B (+17 points absolute) and from 45.2% to 65.9% on Mistral-7B-v0.1 (+46% relative), while affirmative accuracy stayed roughly 91-97%. The best cutoff layer is consistently in the **second half of the network**. They do not test models below ~7B parameters or identify individual shortcut heads (only whole layers).

This is not a Dataset: A Large Negation Benchmark to Challenge LLMs

Garcia-Ferrero et al., EMNLP 2023. A large-scale behavioral benchmark finding LLMs are proficient on affirmative sentences but struggle badly on negated ones, often relying on superficial lexical cues rather than a deep representation of negation. Fine-tuning on negated examples improves in-domain performance but **does not generalize** to new negation forms -- consistent with the structural explanation that "not X" and "X" occur in overlapping training contexts (distributional semantics), making the distinction inherently hard to learn from next-token prediction alone.

Relation to this report's findings

All three sources point to the same two-stage picture this report's plots independently surface in GPT-2 124M: (1) negation IS represented somewhere in the network (consistent with Al Mofael et al.'s layers 4-6), but (2) a separate, later set of attention heads overrides it with an associative shortcut (consistent with Zhou et al.'s late-layer finding, and matching our own L9H9/L10H7/L10H0/L11H10 heads, all concentrated in layers 9-11 -- the back third of GPT-2's 12 layers, directly analogous to their "above 0.5L" cutoff). The present report adds finer granularity than either prior work: individual-head, not just layer-level, logit tracking, and a layer-by-layer crossover table showing exactly where the hijacking token's raw compatibility score first overtakes the correct answer's.

Discussion: A Testable Next Step

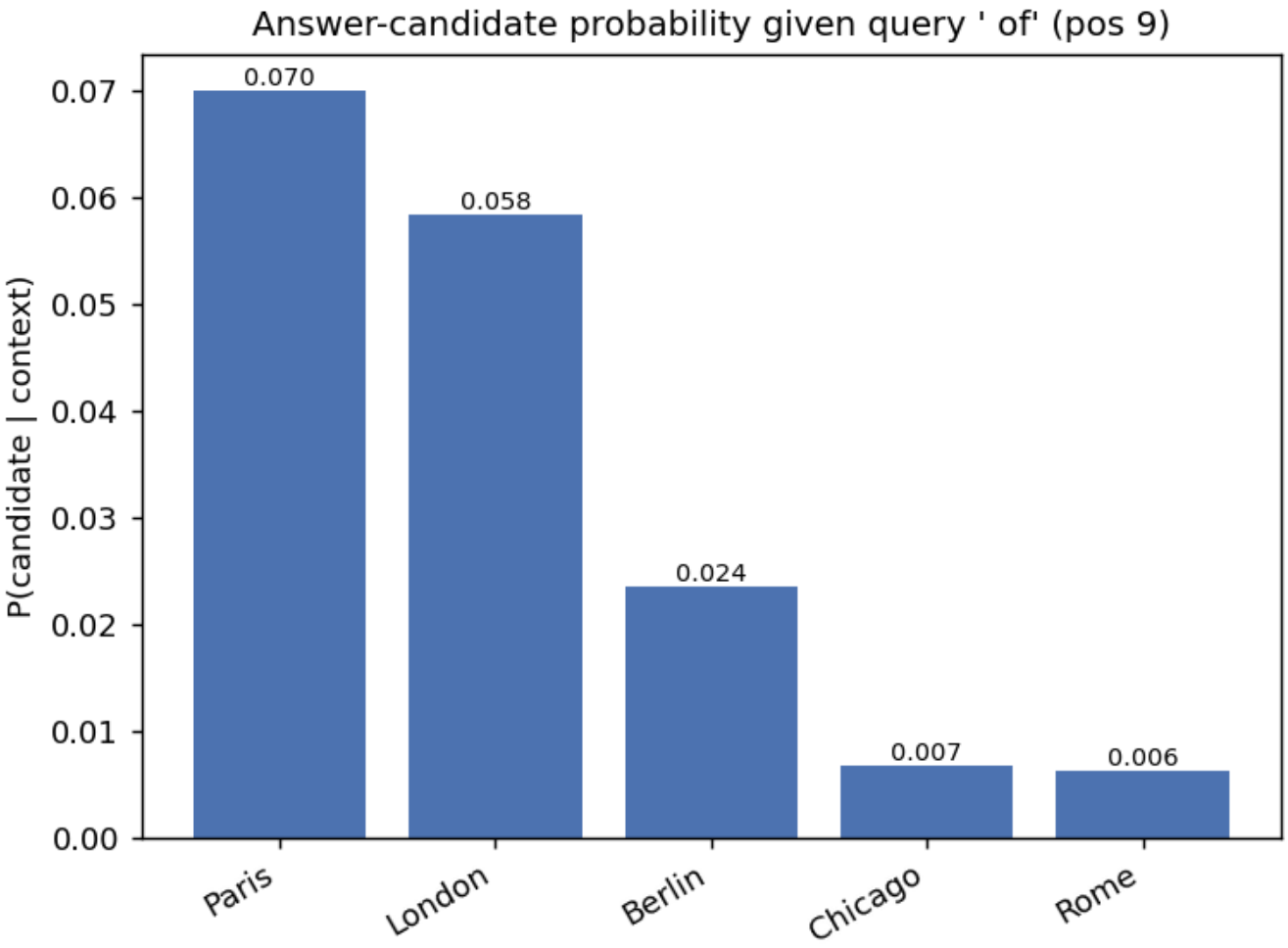
Zhou et al.'s "Cumulative Attention Sink" is directly adaptable to this GPT-2 124M setup using the existing analysis infrastructure: for the query position, modify the attention mask for layers at or above a chosen threshold (candidates: layer 8 or 9, based on the crossover table above) so those layers can only attend to the current token and position 0, then re-run `candidate_probabilities` to check whether Paris's probability recovers relative to Chicago's. This would move the finding in this report from strongly correlational (logit rises, weight rises, wrong answer wins) to causally demonstrated (suppressing the late-layer logit rise measurably restores the correct answer) -- not yet implemented as of this report.

Appendix: Full Plot Log

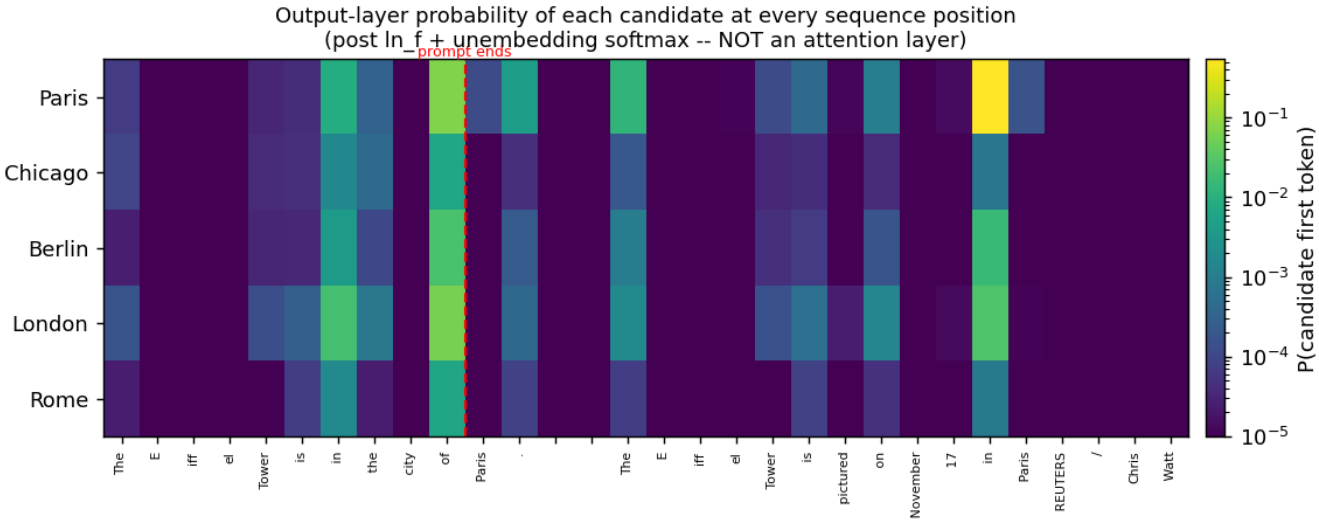
Every plot generated by src/attention_scores.py for all 5 cases, in full. 8 plot types per case.

Case 0: The Eiffel Tower is in the city of

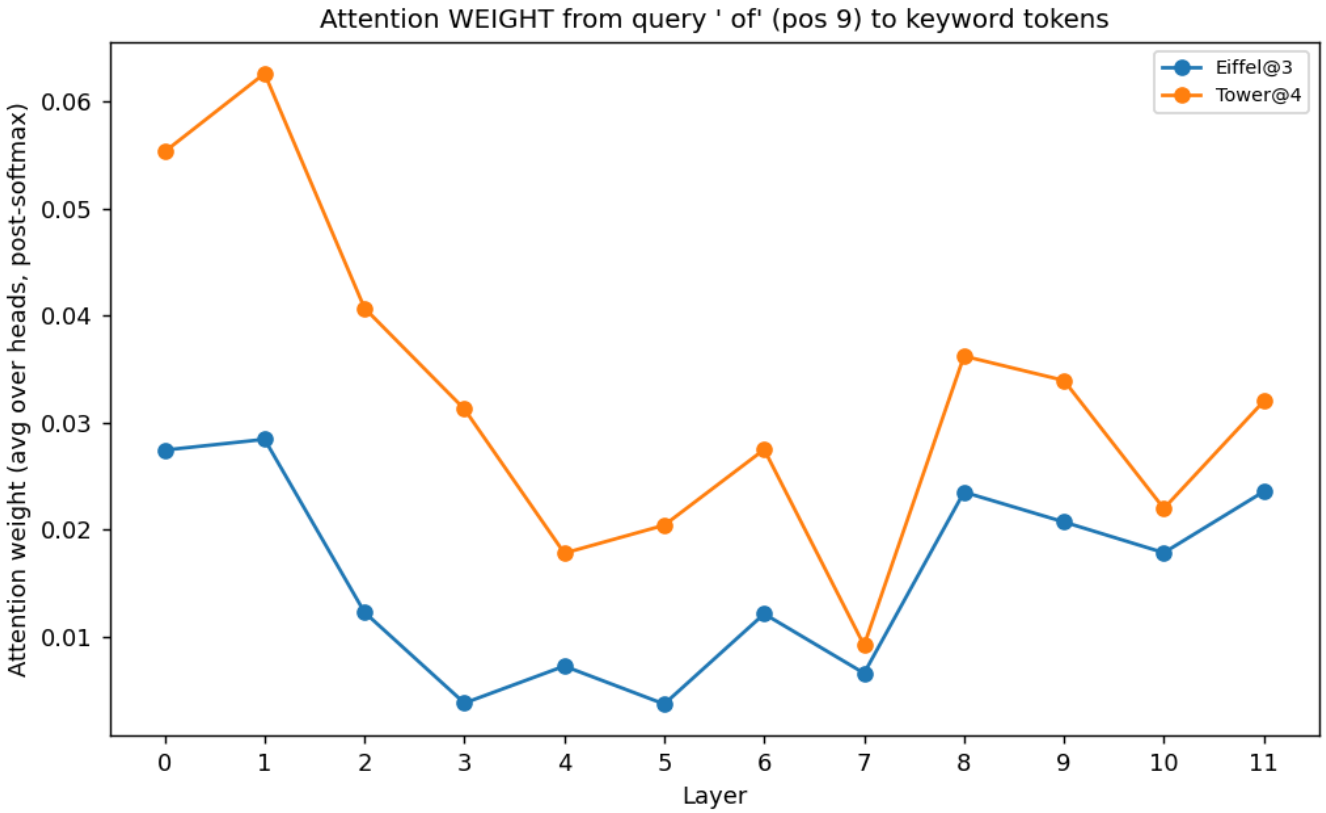
Generated: *Paris.* | baseline, no negation



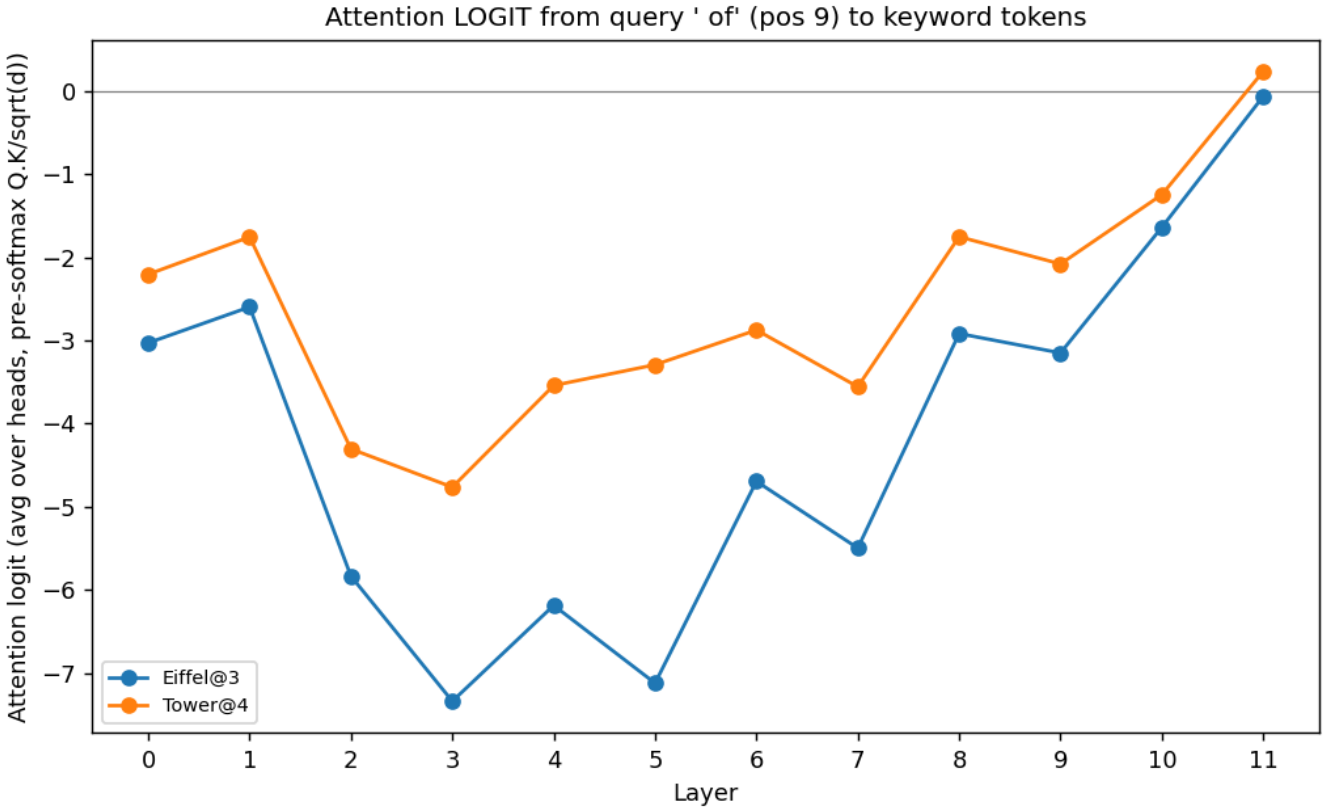
Case 0 -- Candidate probability (output layer) (candidate_probabilities.png)



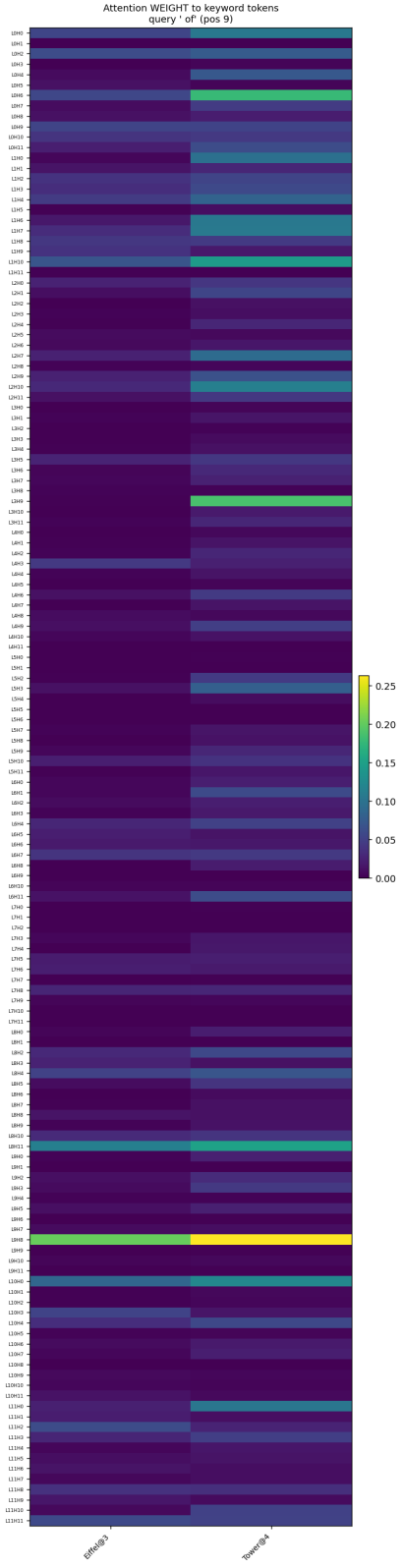
Case 0 -- Output-layer probability across full sequence (output_layer_heatmap.png)



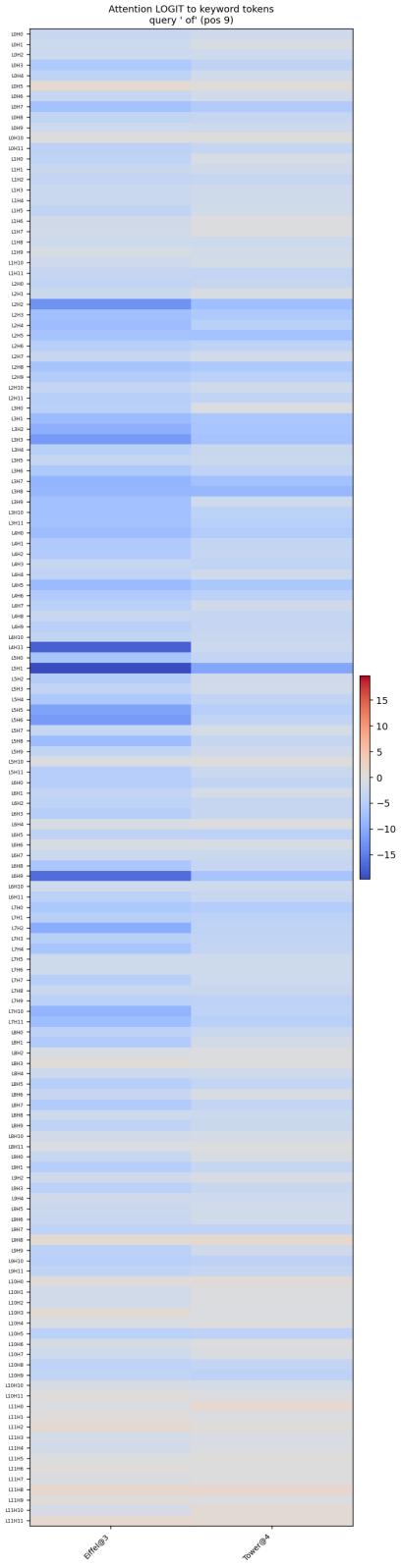
Case 0 -- Keyword attention WEIGHT by layer (keyword_attention_by_layer.png)



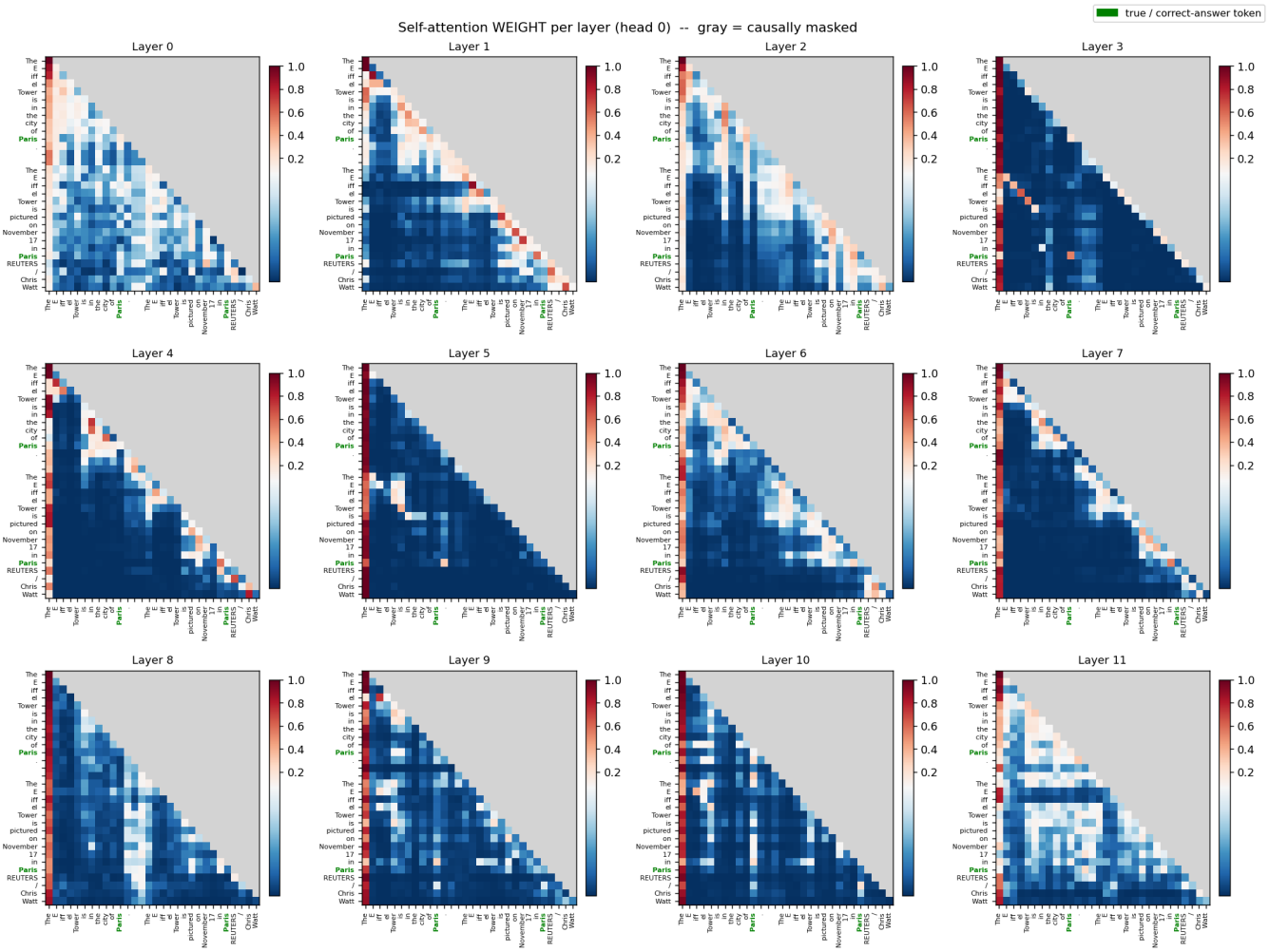
Case 0 -- Keyword attention LOGIT by layer (keyword_attention_logits_by_layer.png)



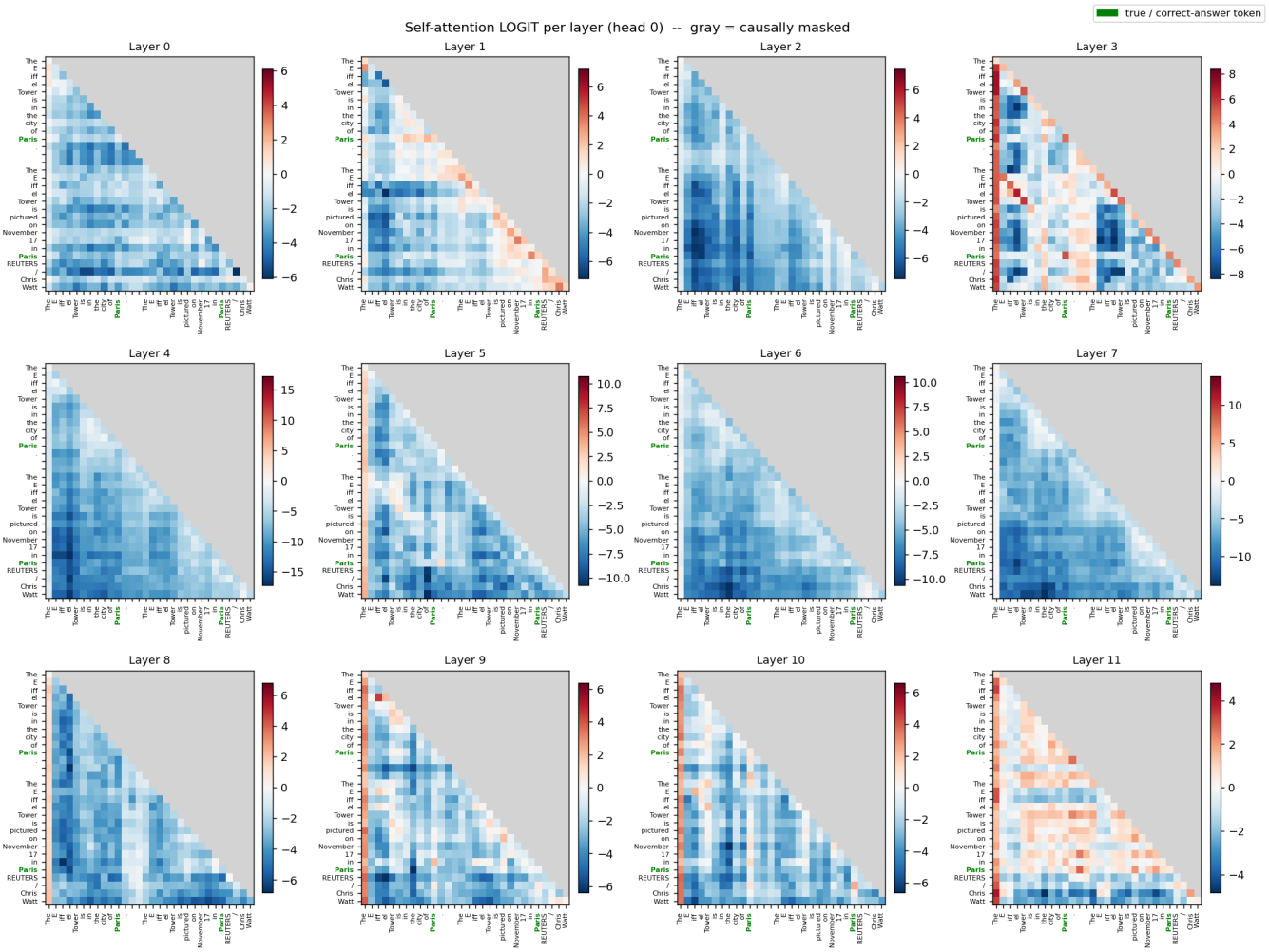
Case 0 -- Keyword attention WEIGHT by (layer, head) (keyword_attention_heads.png)



Case 0 -- Keyword attention LOGIT by (layer, head) (keyword_attention_logits_heads.png)



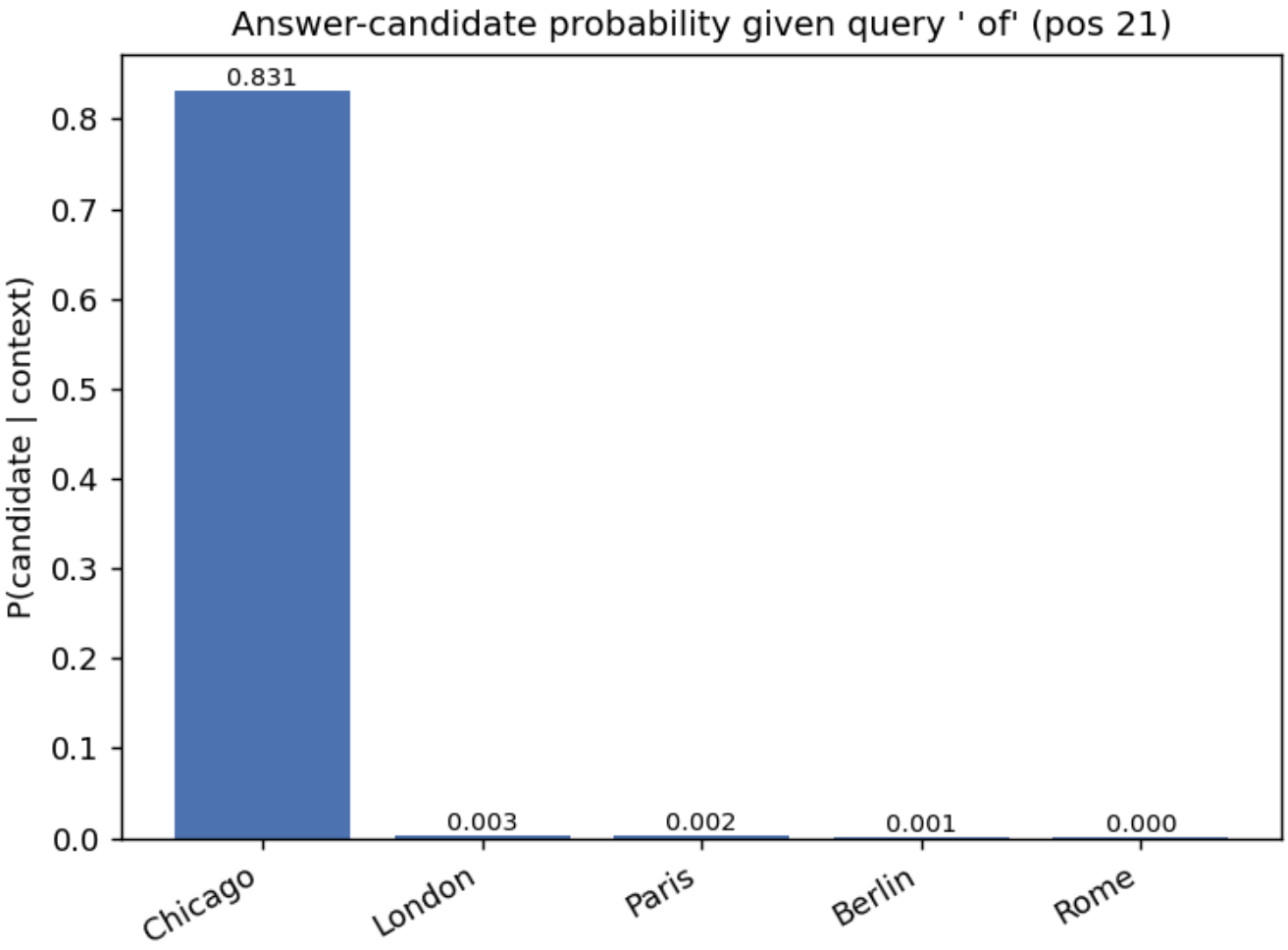
Case 0 -- Full per-layer self-attention WEIGHT grid (head 0) (attention_heatmaps.png)



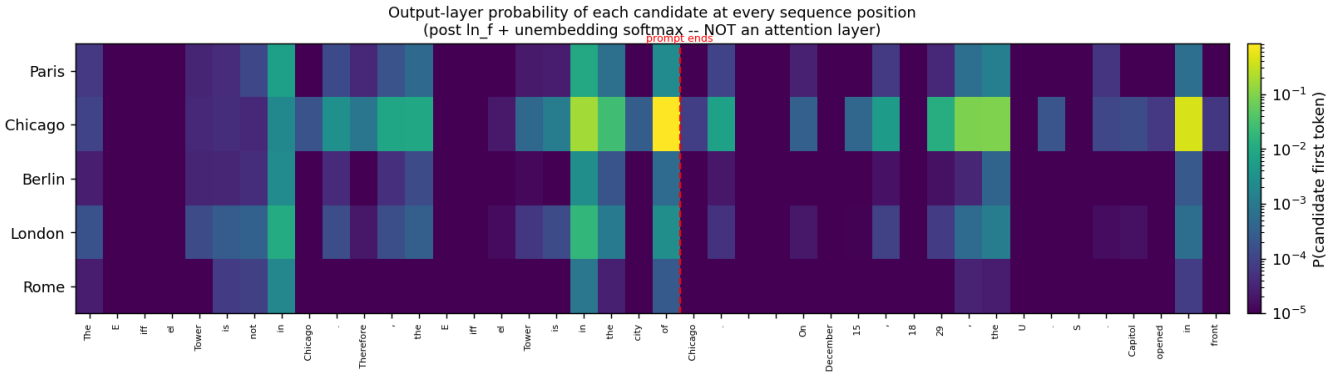
Case 0 -- Full per-layer self-attention LOGIT grid (head 0) (attention_logit_heatmaps.png)

Case 1: The Eiffel Tower is not in Chicago. Therefore, the Eiffel Tower is in the city of

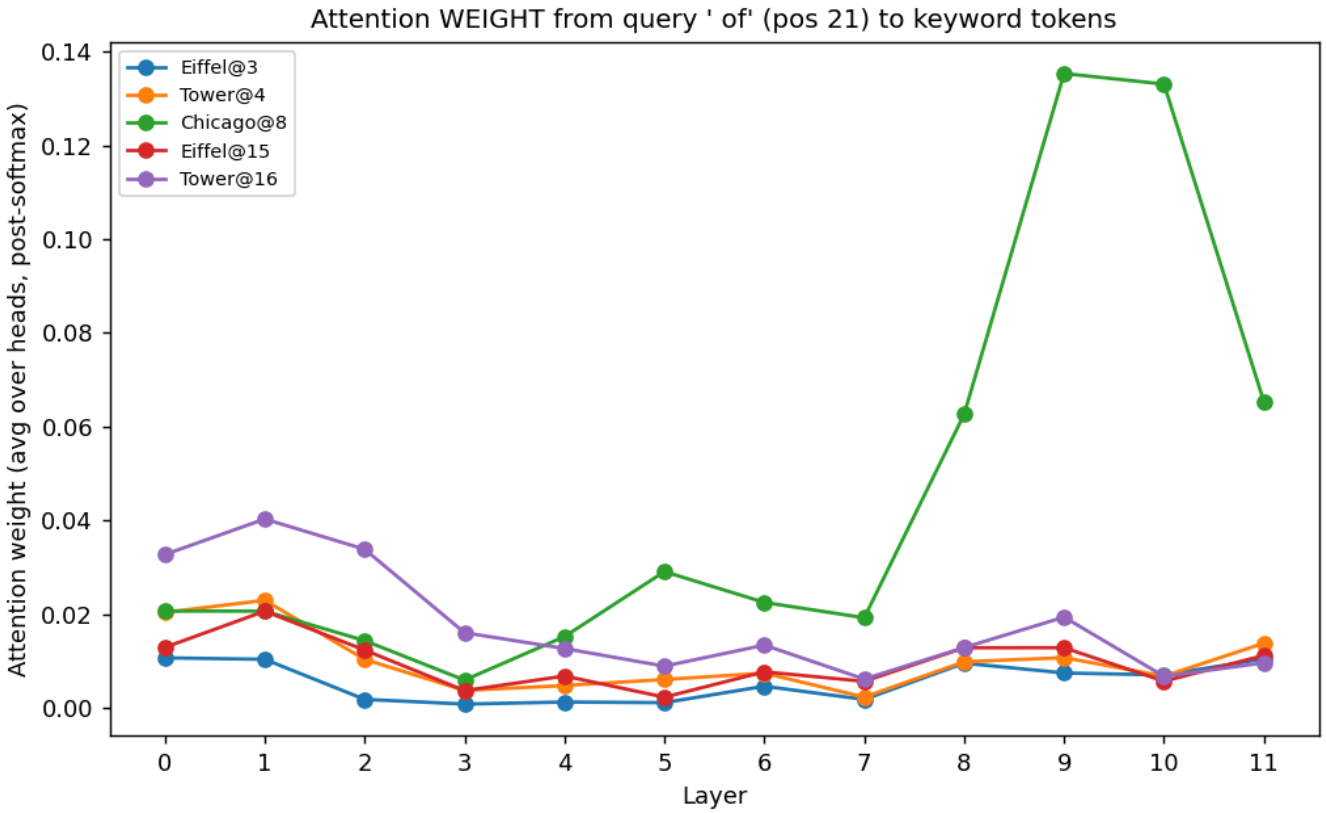
Generated: *Chicago.* | single negation



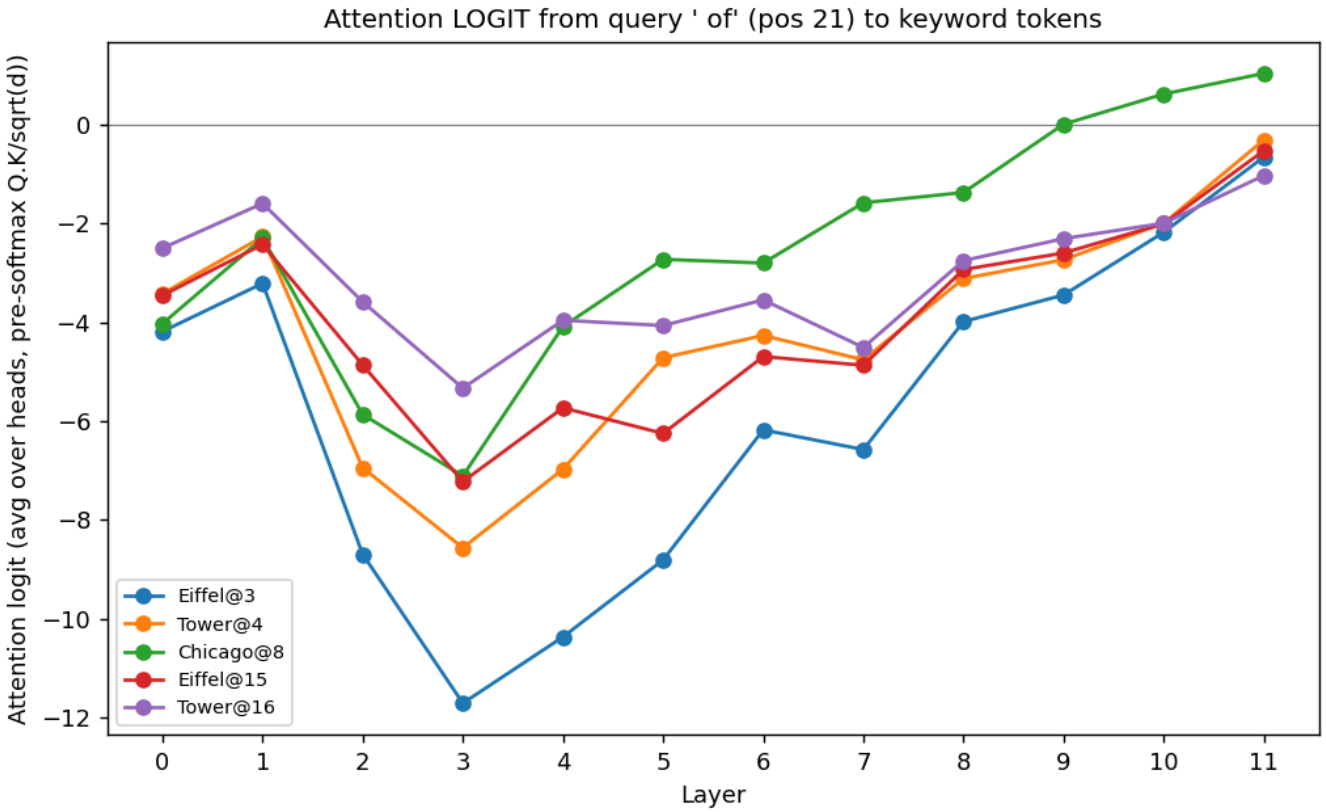
Case 1 -- Candidate probability (output layer) (candidate_probabilities.png)



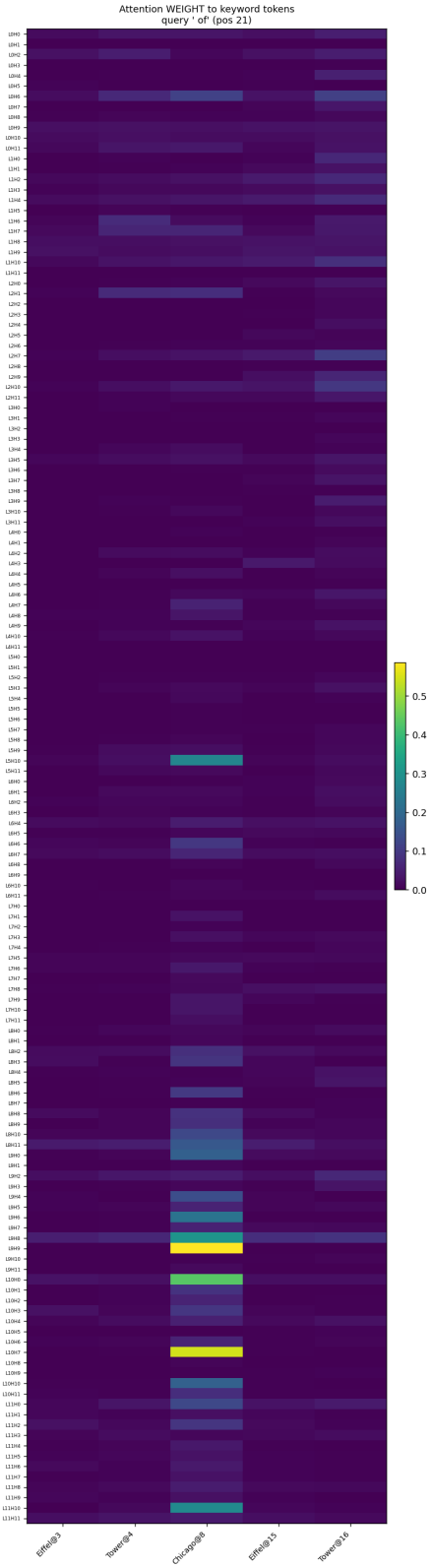
Case 1 -- Output-layer probability across full sequence (output_layer_heatmap.png)



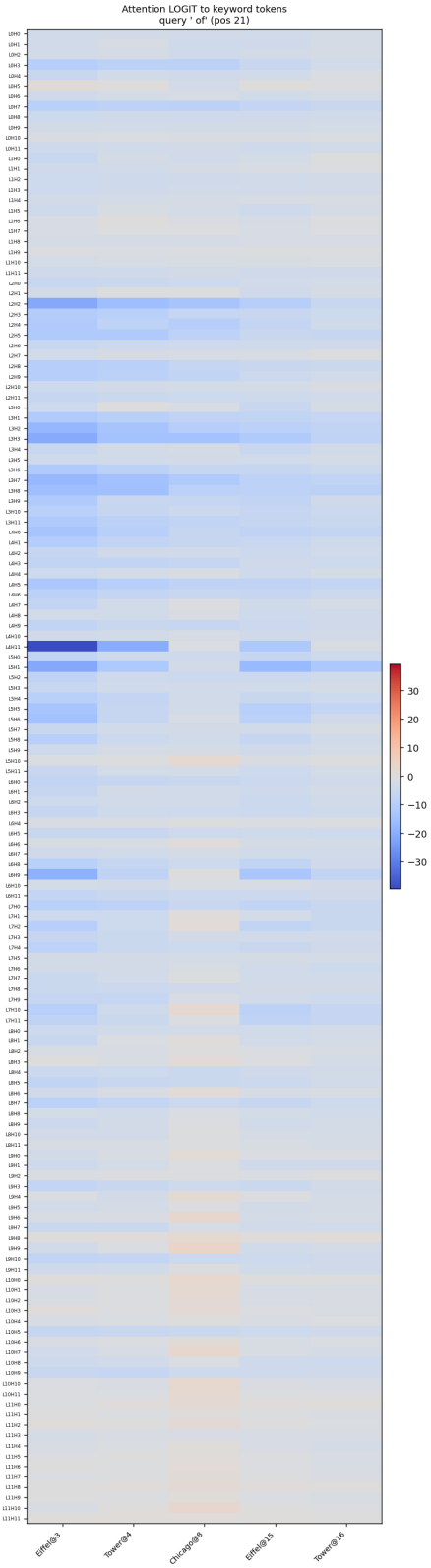
Case 1 -- Keyword attention WEIGHT by layer (keyword_attention_by_layer.png)



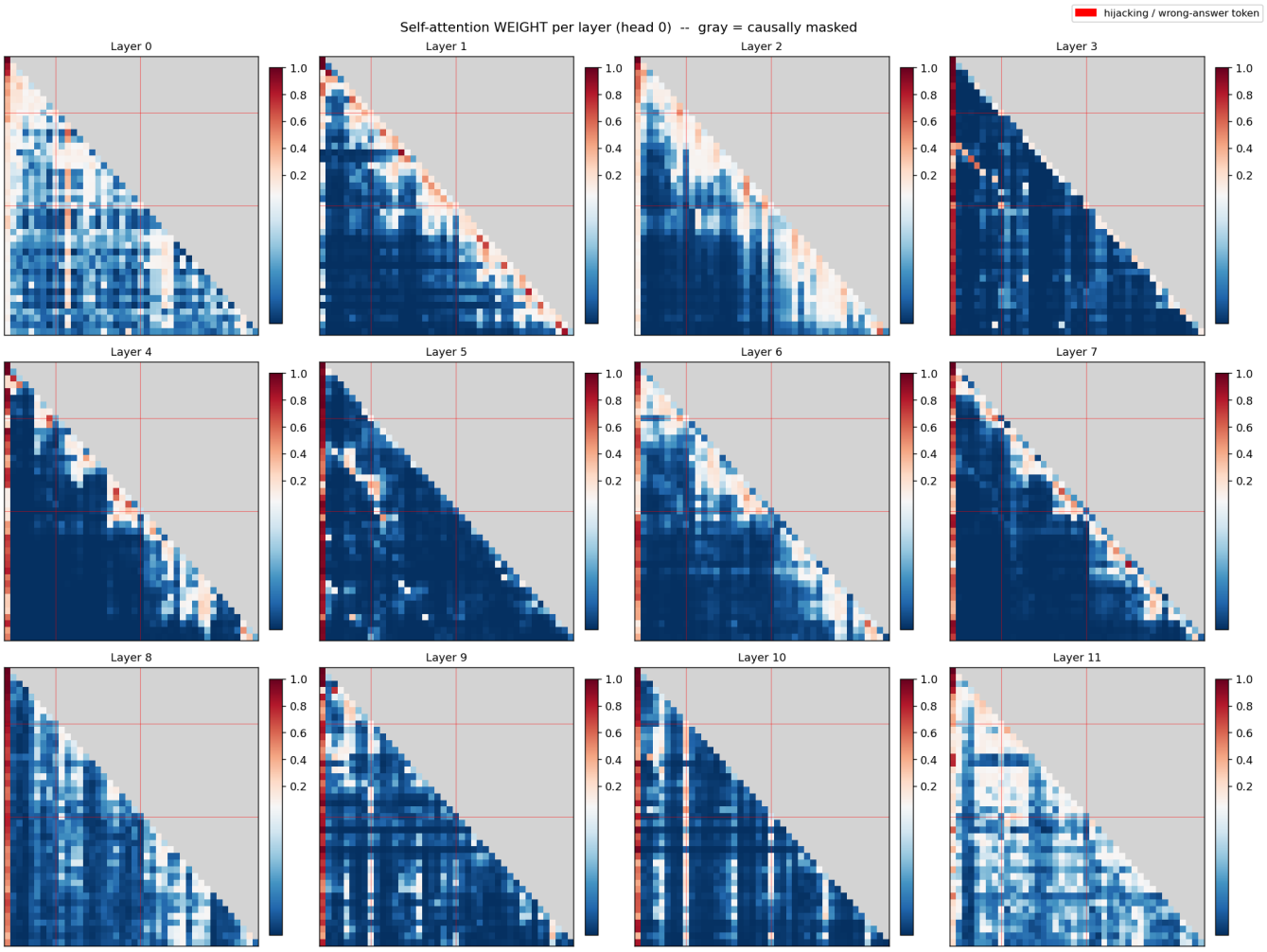
Case 1 -- Keyword attention LOGIT by layer (keyword_attention_logits_by_layer.png)



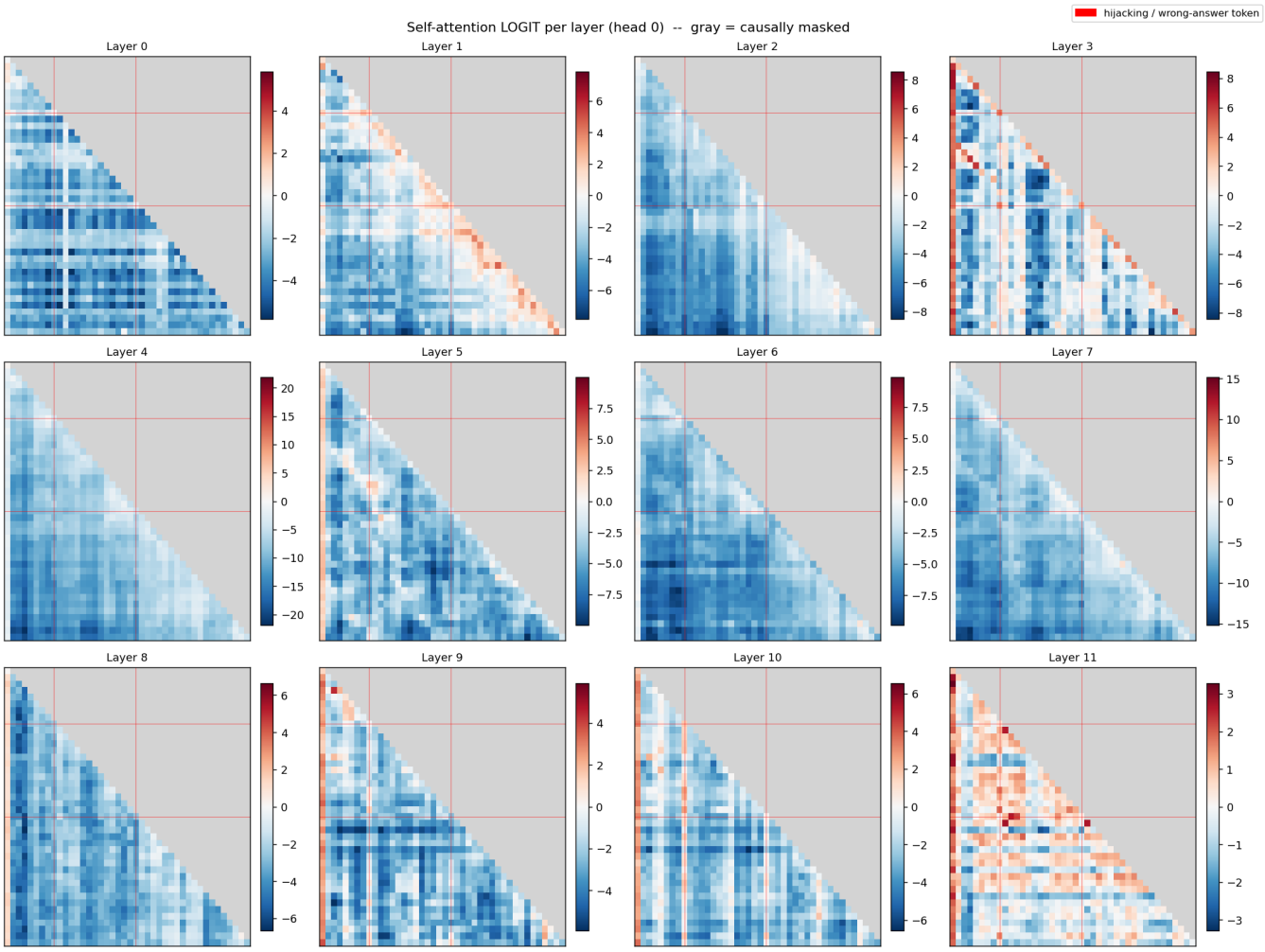
Case 1 -- Keyword attention WEIGHT by (layer, head) (keyword_attention_heads.png)



Case 1 -- Keyword attention LOGIT by (layer, head) (keyword_attention_logits_heads.png)



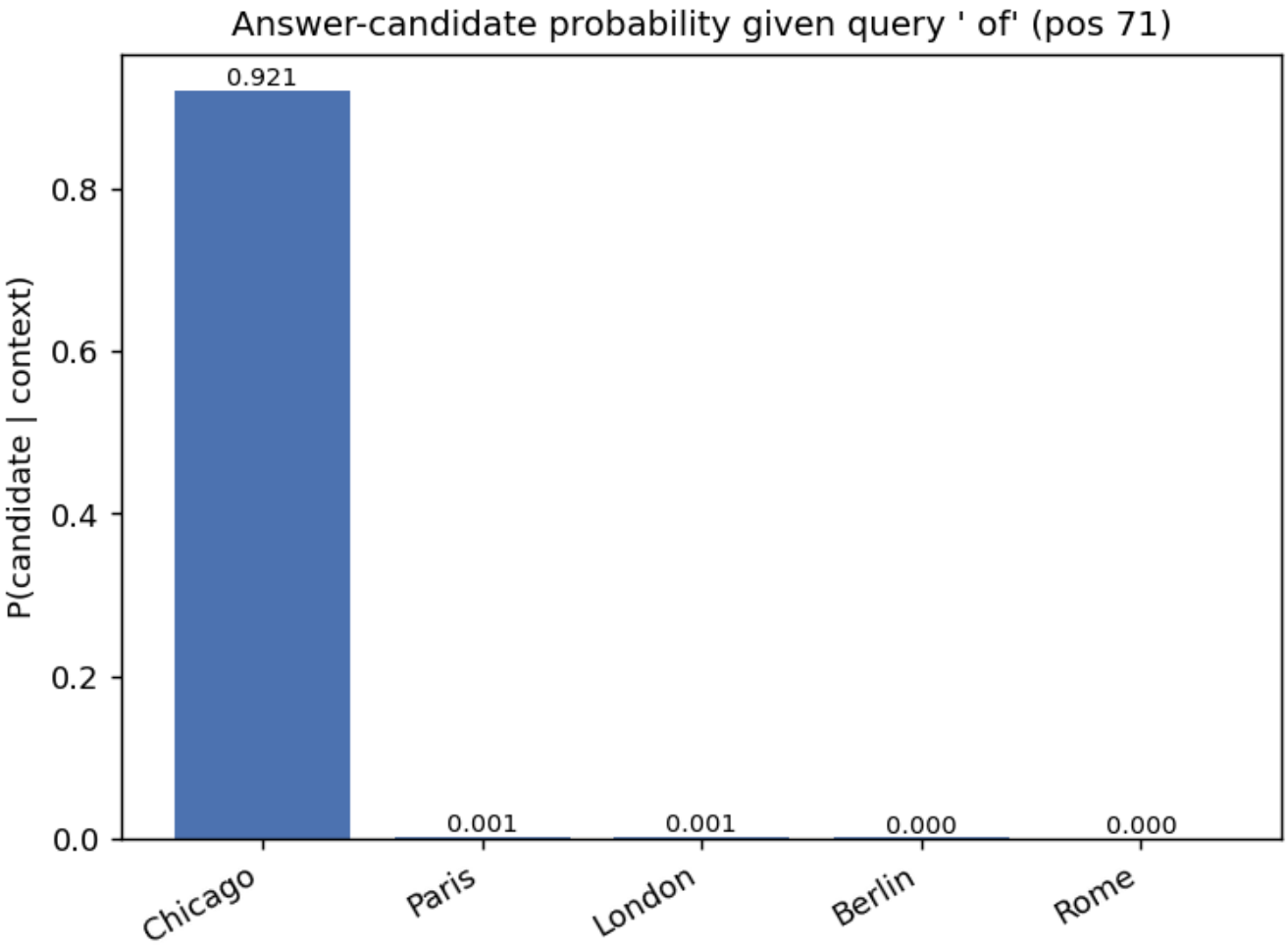
Case 1 -- Full per-layer self-attention WEIGHT grid (head 0) (attention_heatmaps.png)



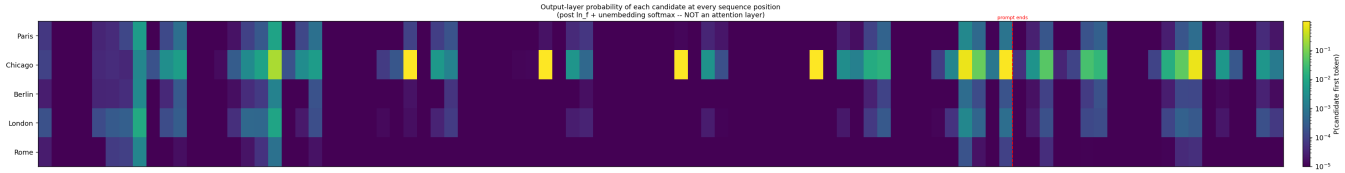
Case 1 -- Full per-layer self-attention LOGIT grid (head 0) (attention_logit_heatmaps.png)

Case 2: (6x) The Eiffel Tower is not in Chicago. ... Therefore, the Eiffel Tower is in the city of

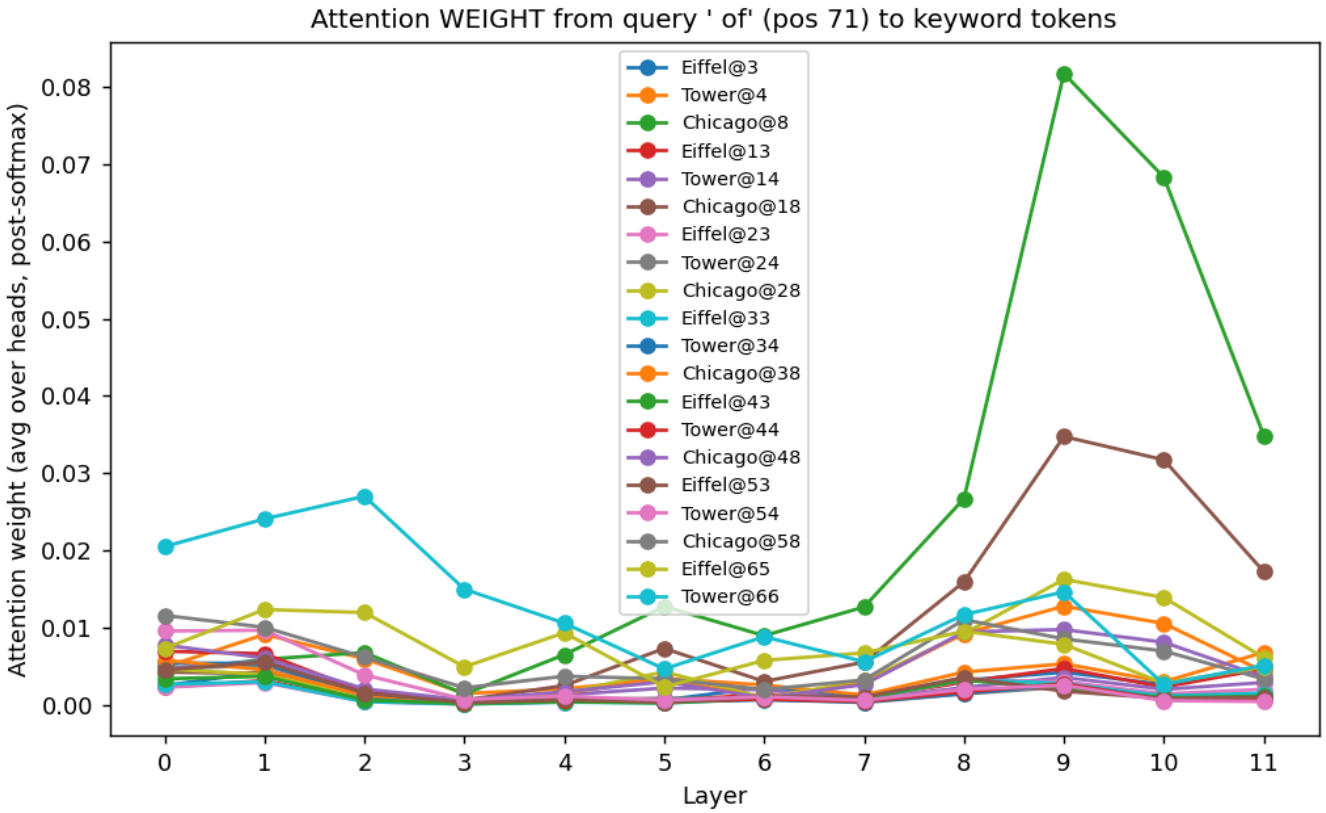
Generated: *Chicago.* | negation repeated 6x



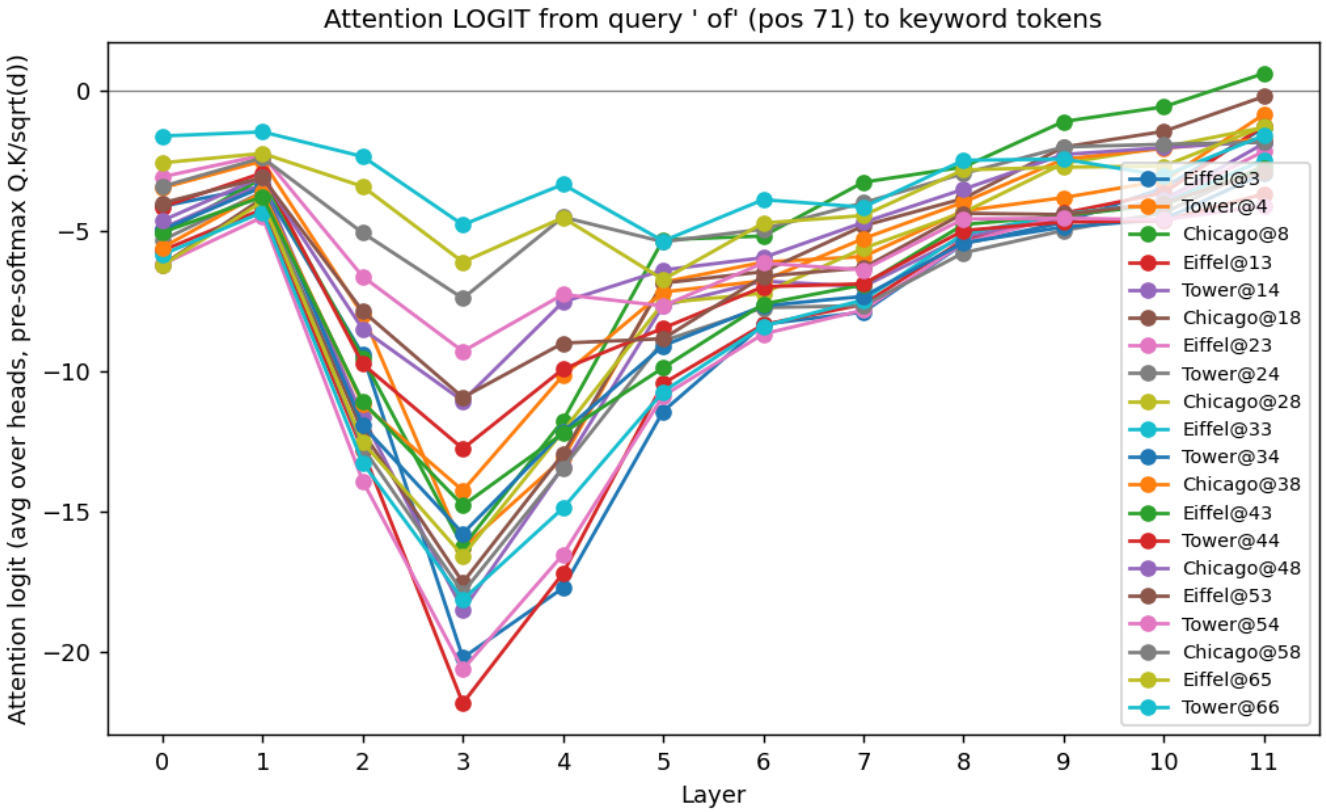
Case 2 -- Candidate probability (output layer) (candidate_probabilities.png)



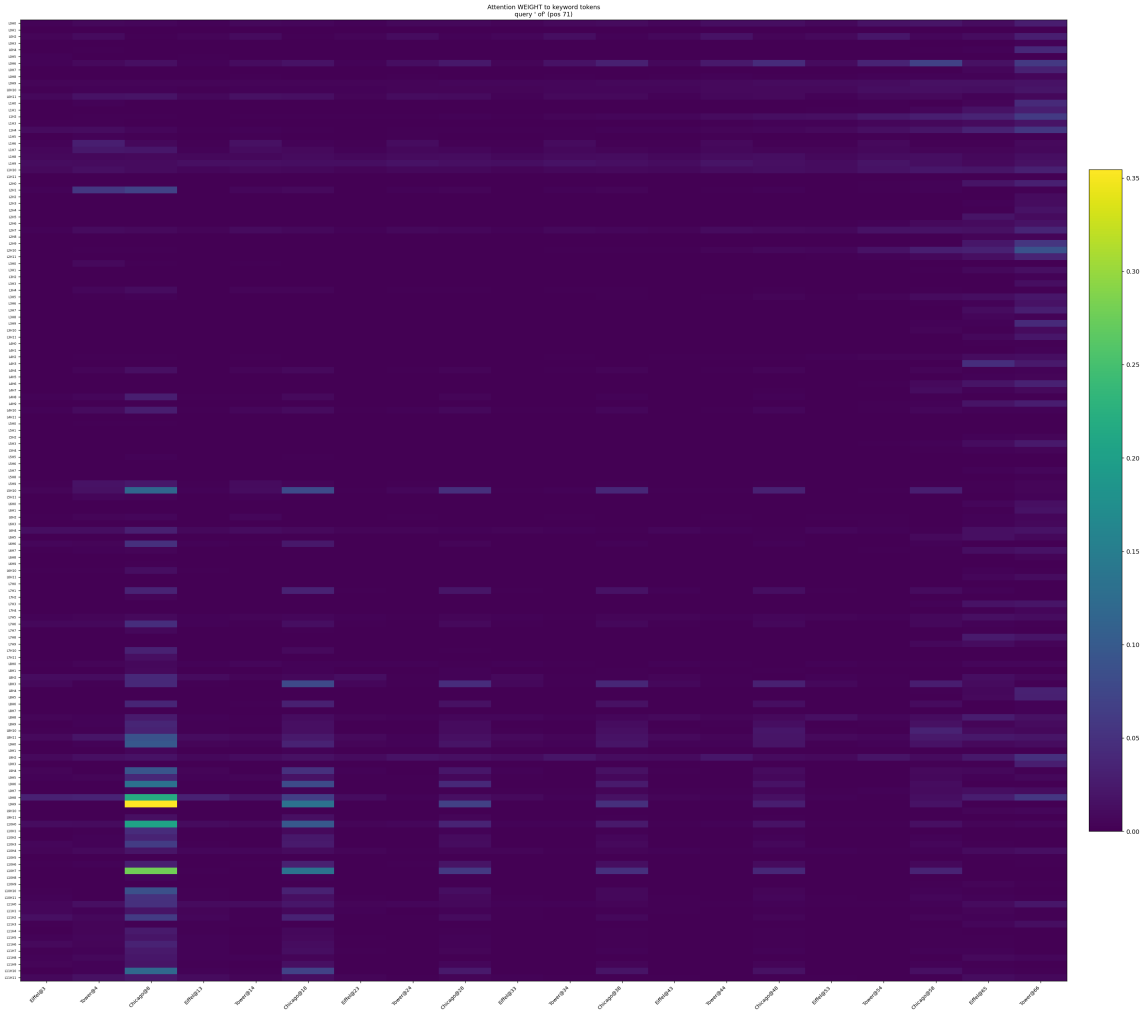
Case 2 -- Output-layer probability across full sequence (output_layer_heatmap.png)



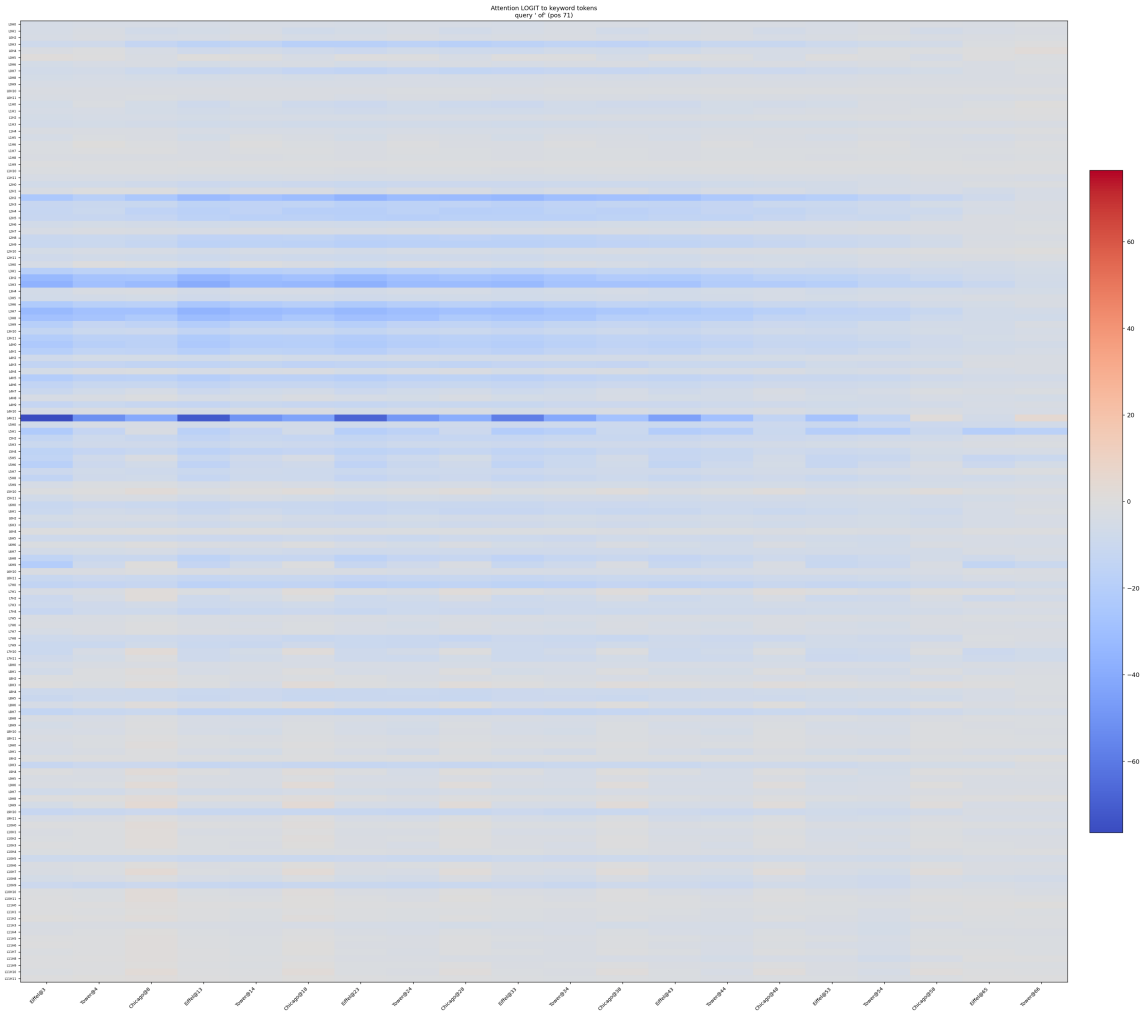
Case 2 -- Keyword attention WEIGHT by layer (keyword_attention_by_layer.png)



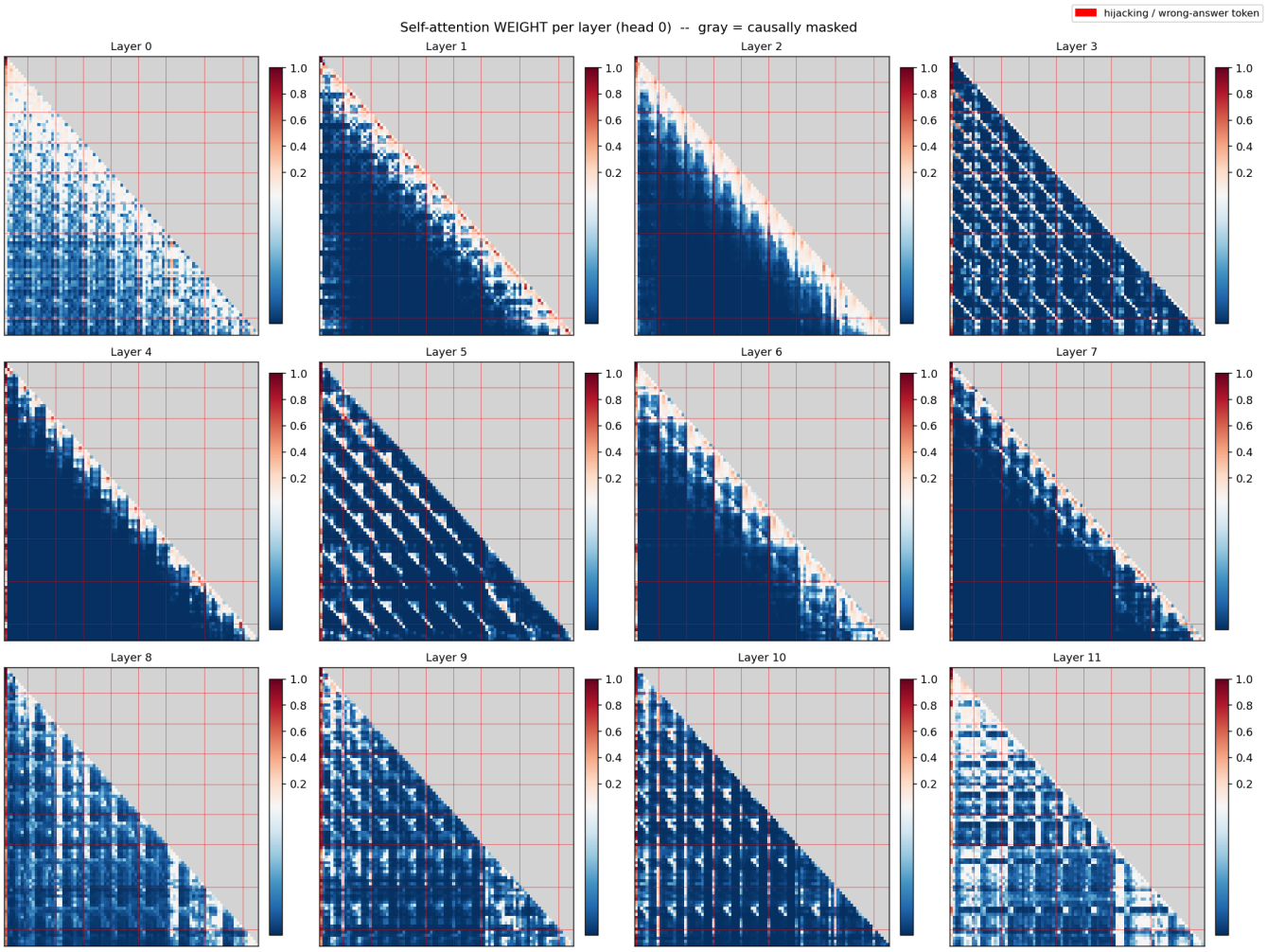
Case 2 -- Keyword attention LOGIT by layer (keyword_attention_logits_by_layer.png)



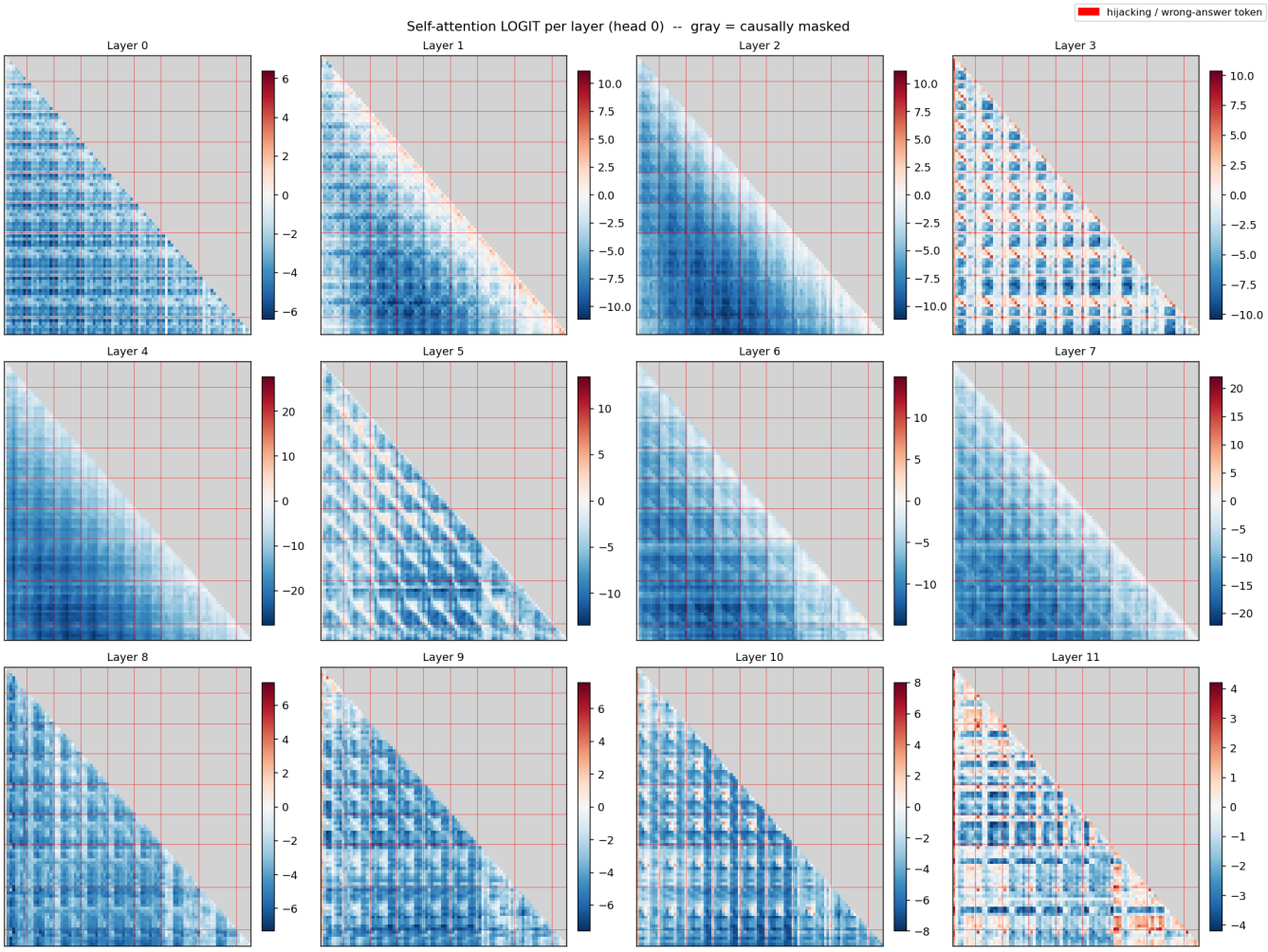
Case 2 -- Keyword attention WEIGHT by (layer, head) (keyword_attention_heads.png)



Case 2 -- Keyword attention LOGIT by (layer, head) (keyword_attention_logits_heads.png)



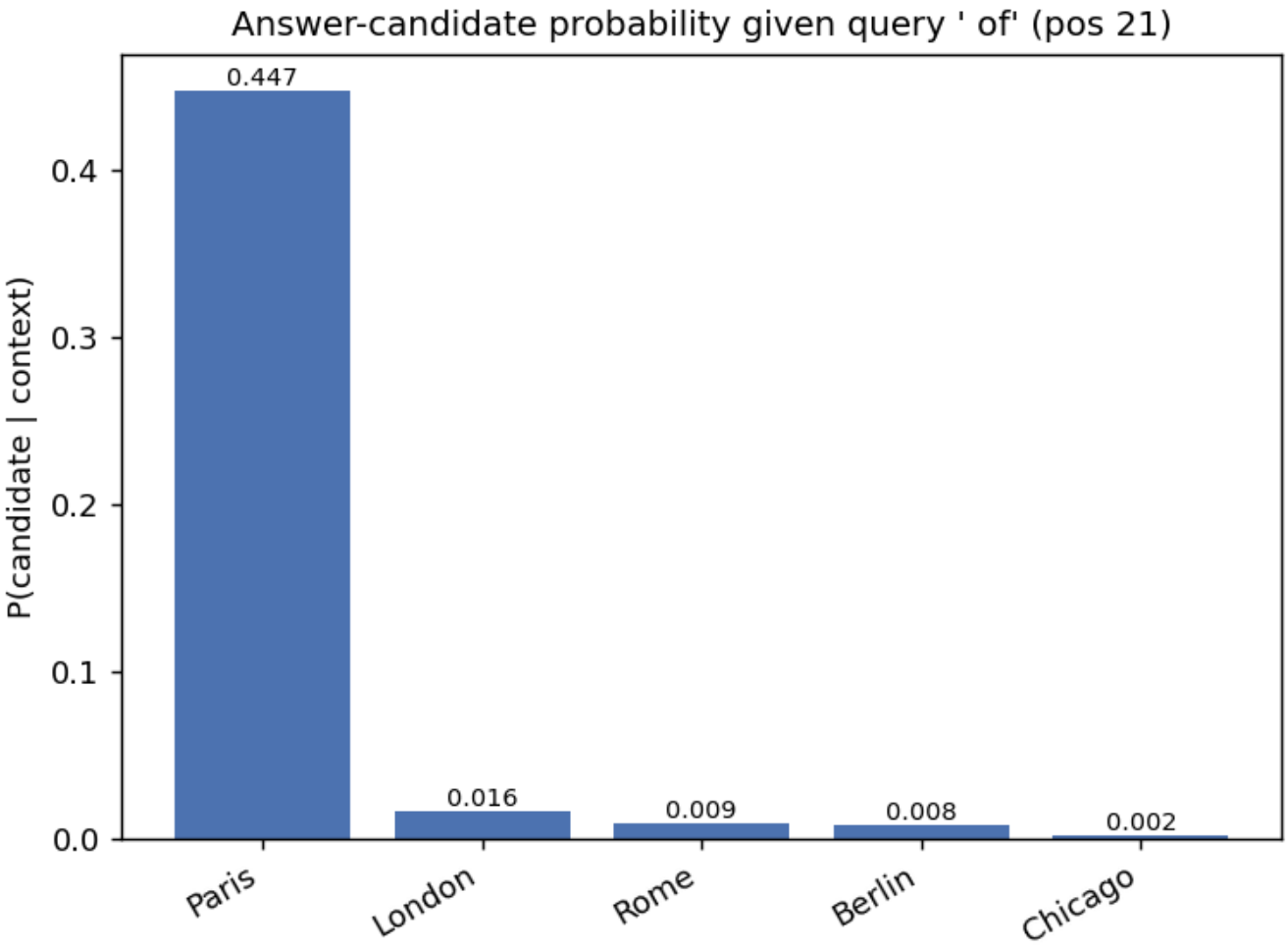
Case 2 -- Full per-layer self-attention WEIGHT grid (head 0) (attention_heatmaps.png)



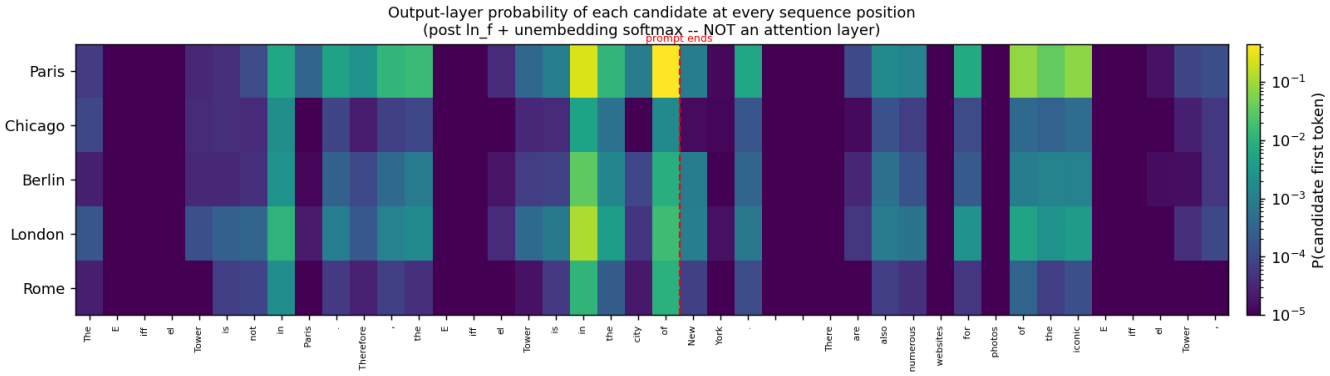
Case 2 -- Full per-layer self-attention LOGIT grid (head 0) (attention_logit_heatmaps.png)

Case 3: The Eiffel Tower is not in Paris. Therefore, the Eiffel Tower is in the city of

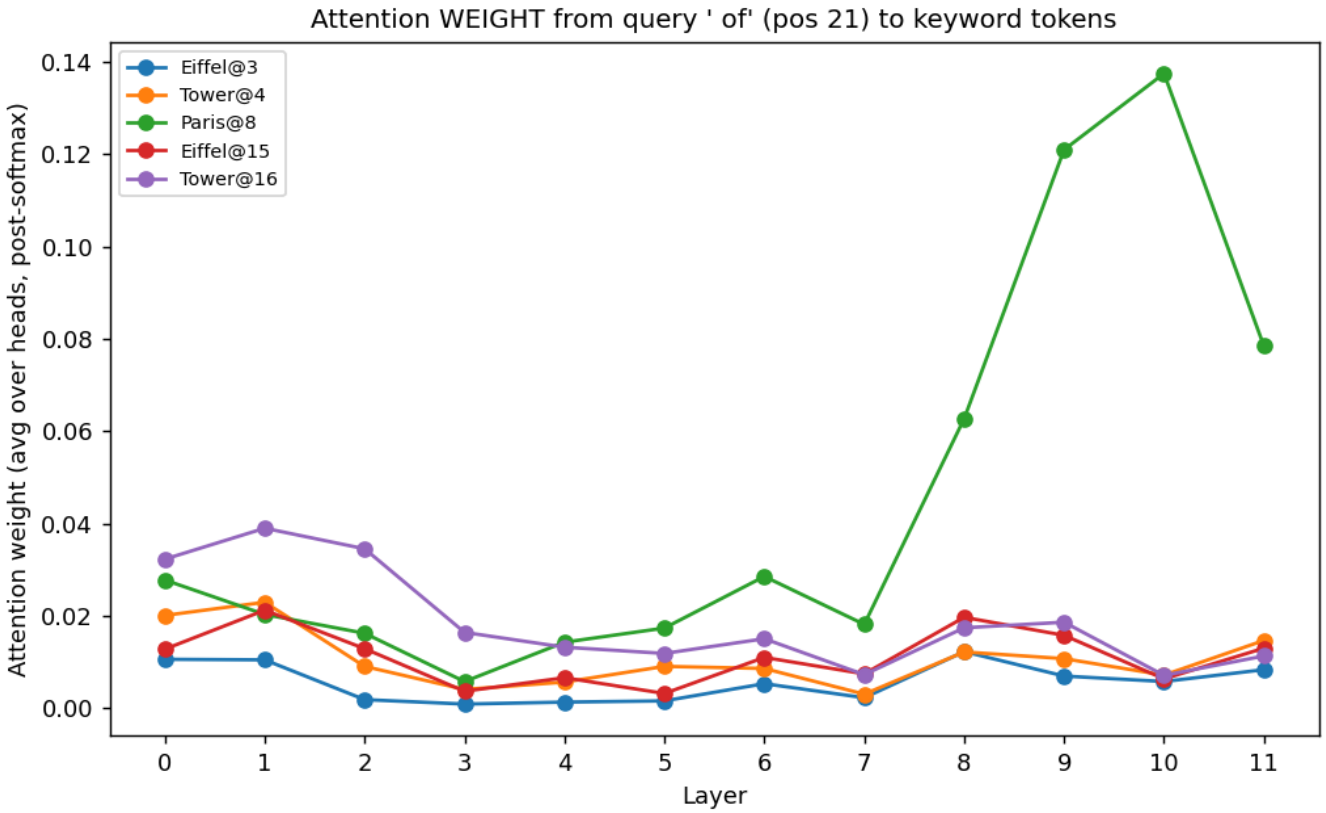
Generated: *New York.* | entities swapped: Paris negated



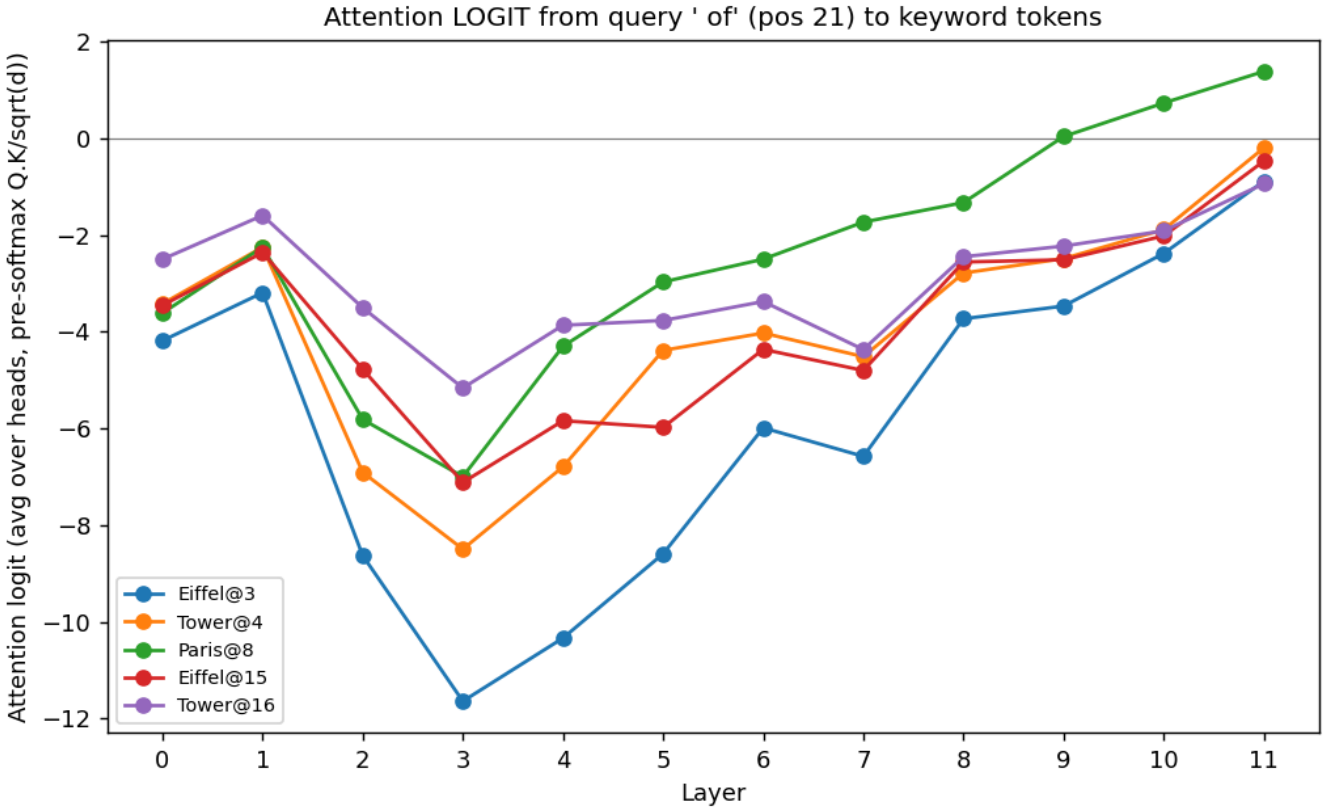
Case 3 -- Candidate probability (output layer) (candidate_probabilities.png)



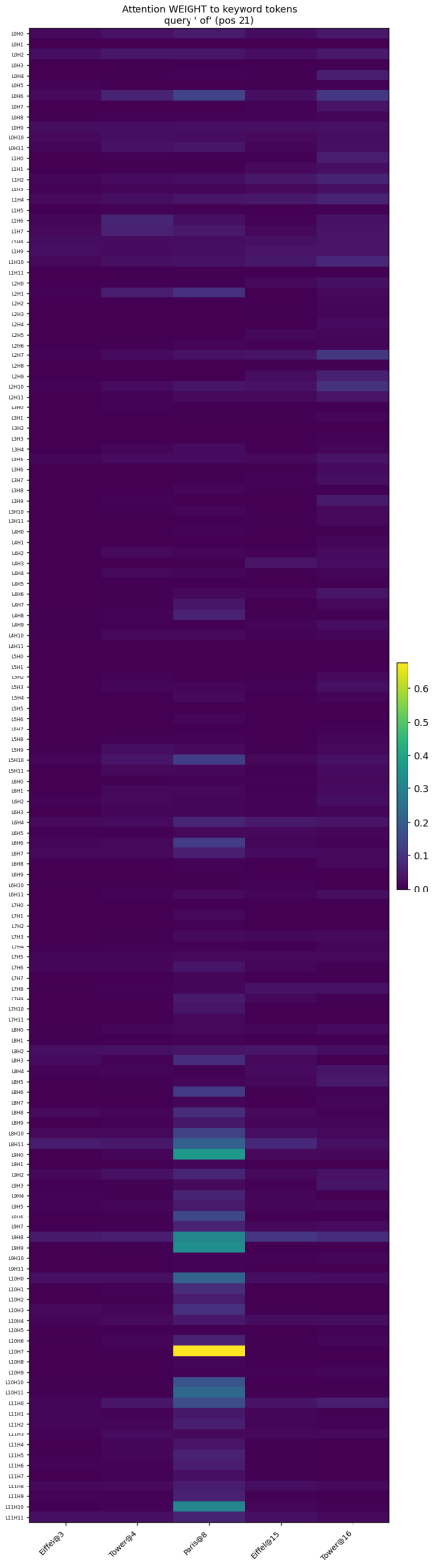
Case 3 -- Output-layer probability across full sequence (output_layer_heatmap.png)



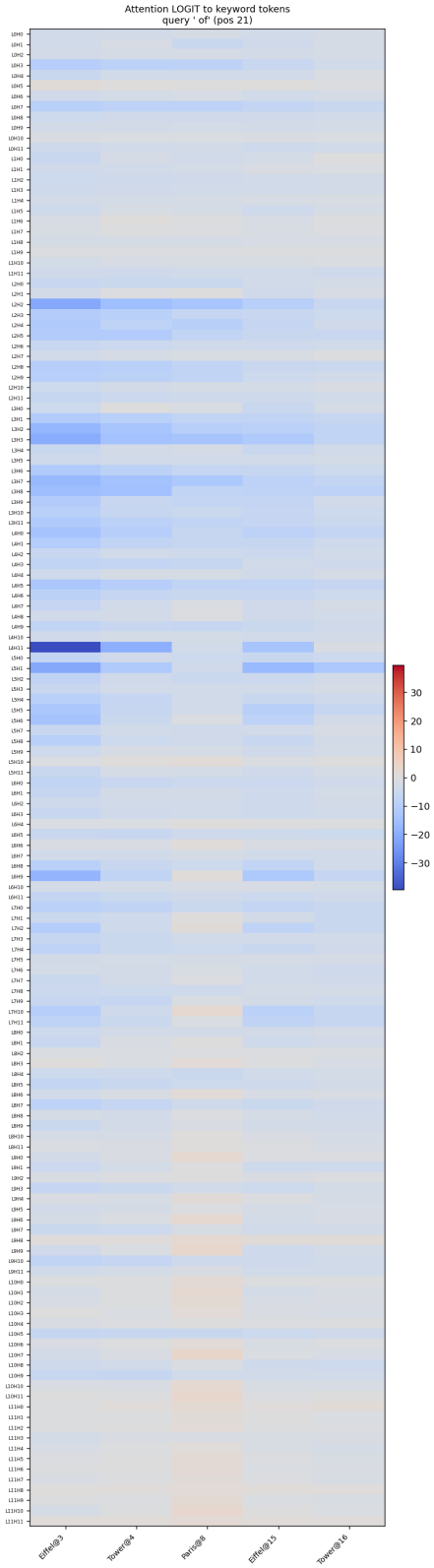
Case 3 -- Keyword attention WEIGHT by layer (keyword_attention_by_layer.png)



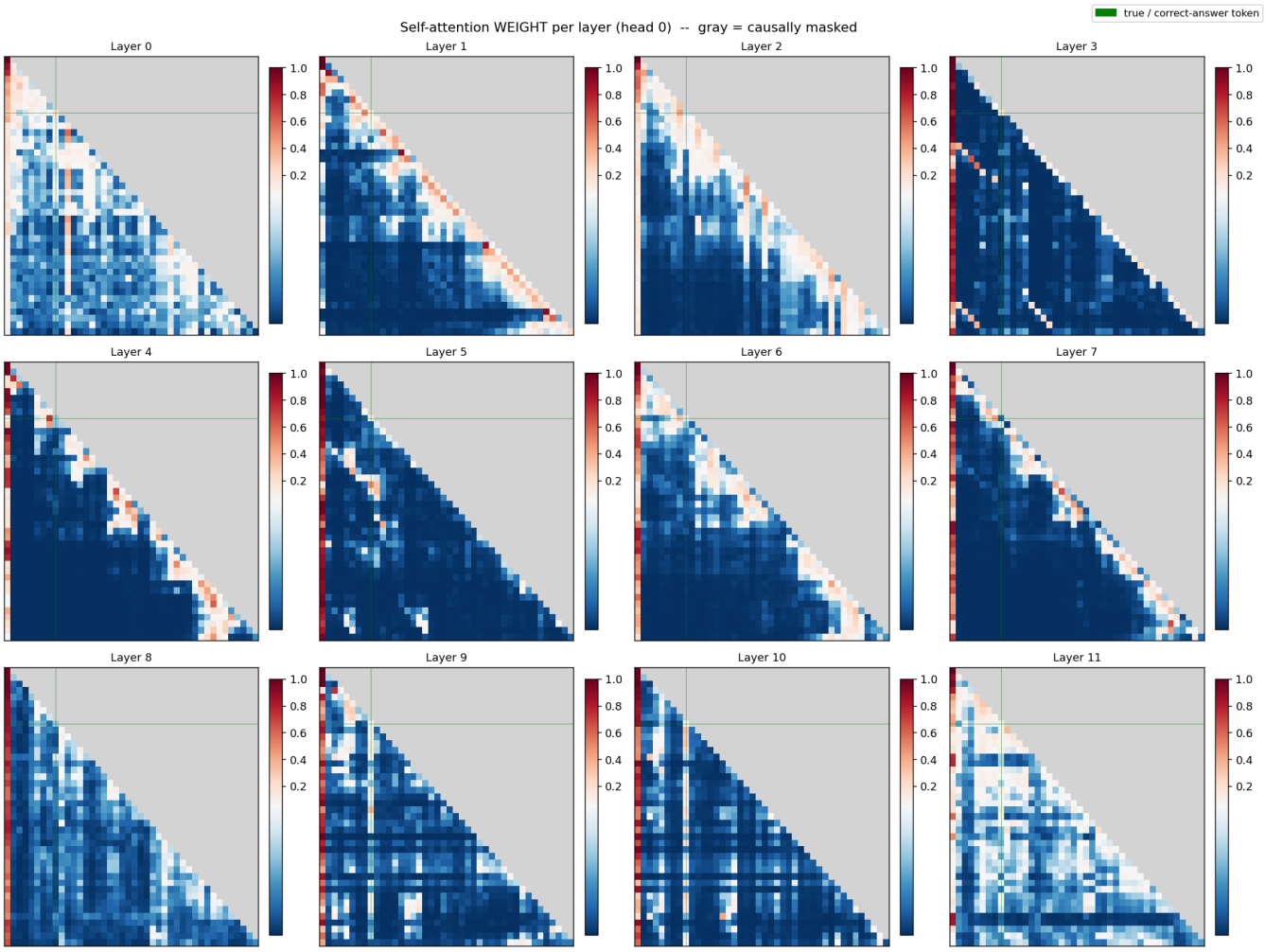
Case 3 -- Keyword attention LOGIT by layer (keyword_attention_logits_by_layer.png)



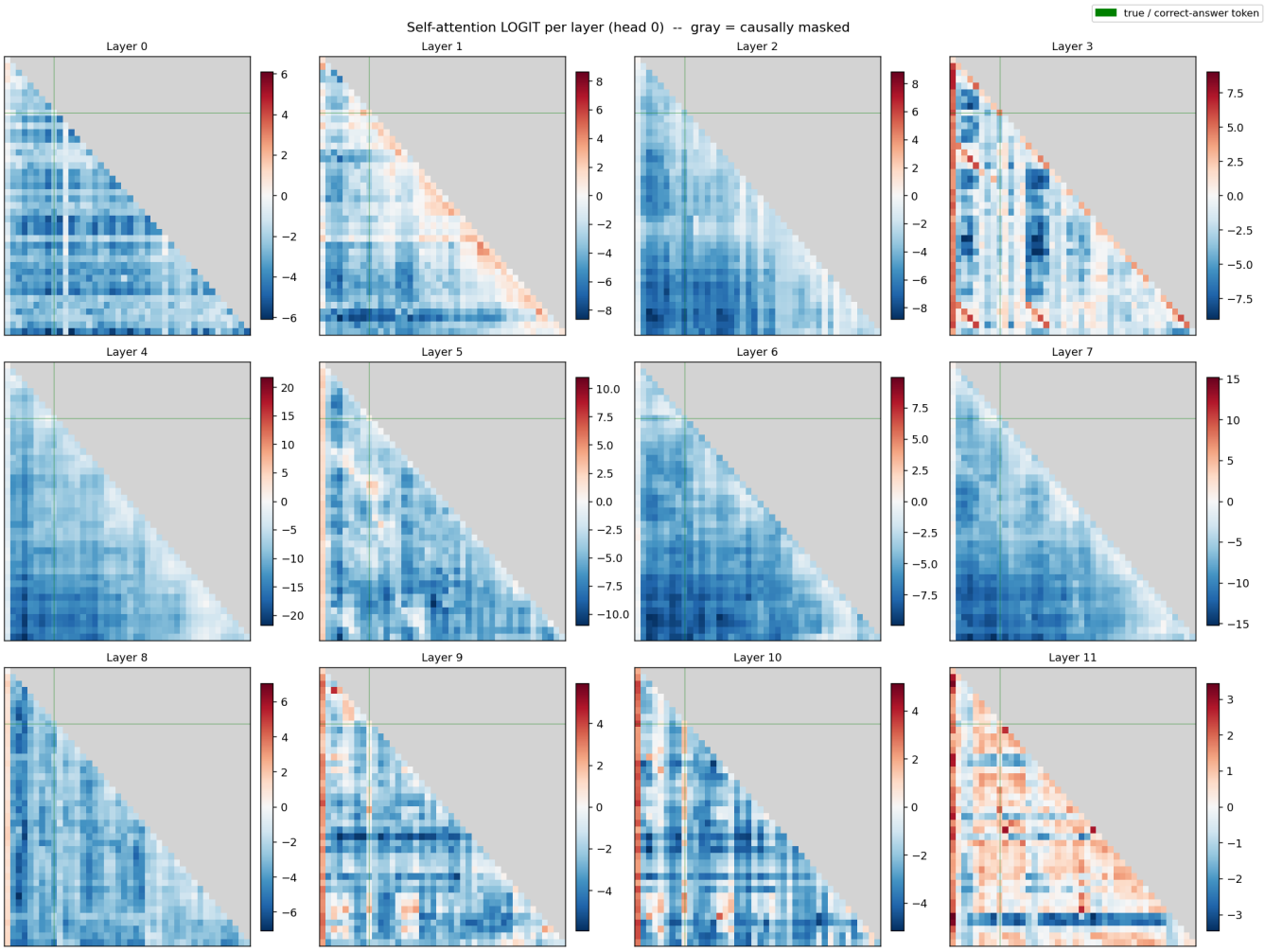
Case 3 -- Keyword attention WEIGHT by (layer, head) (keyword_attention_heads.png)



Case 3 -- Keyword attention LOGIT by (layer, head) (keyword_attention_logits_heads.png)



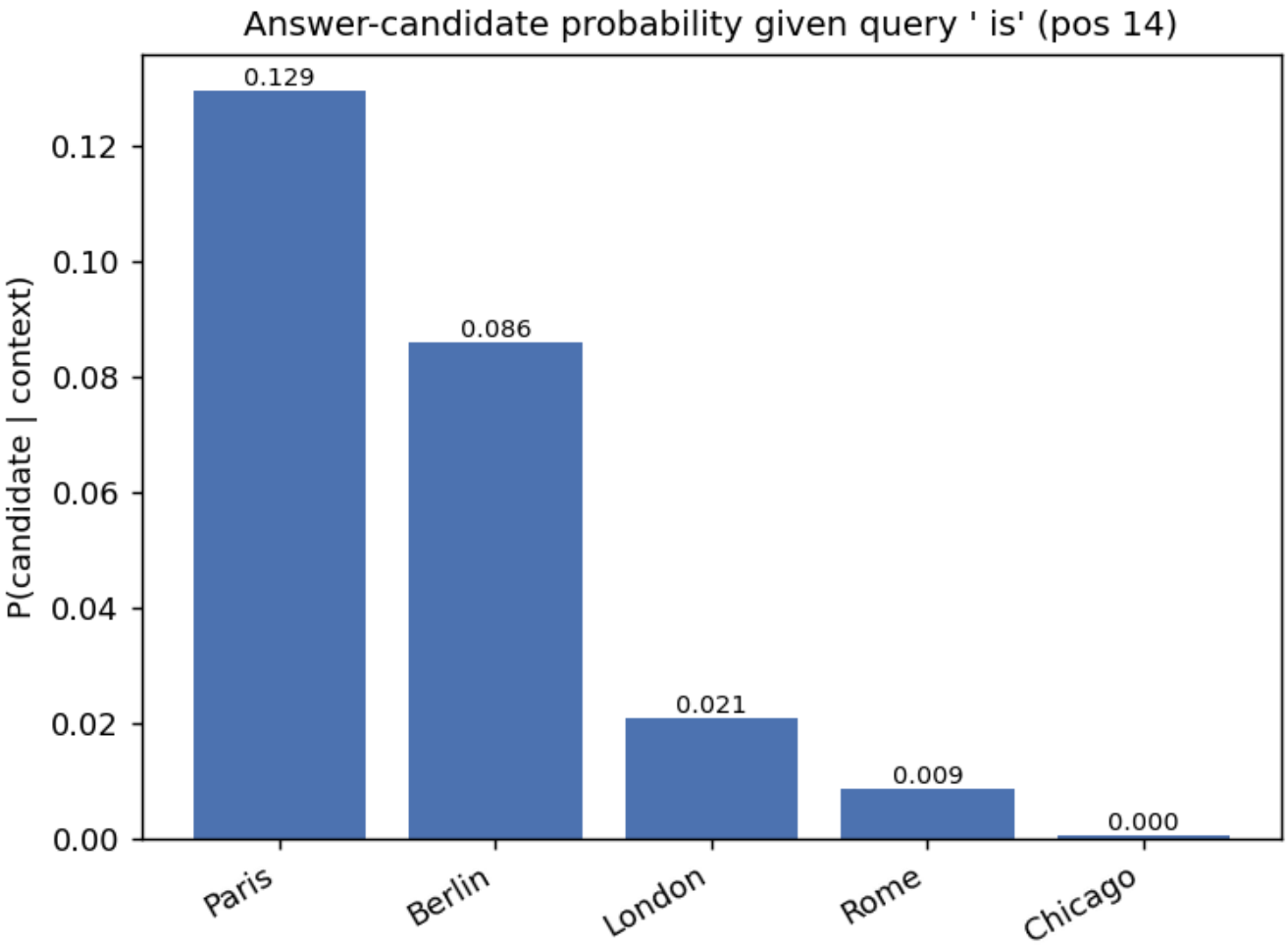
Case 3 -- Full per-layer self-attention WEIGHT grid (head 0) (attention_heatmaps.png)



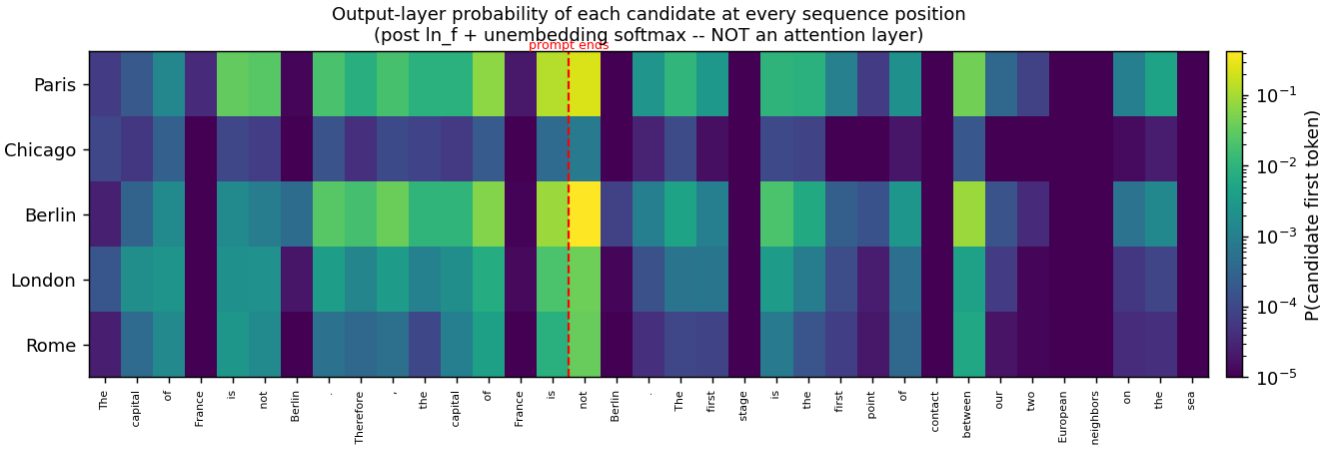
Case 3 -- Full per-layer self-attention LOGIT grid (head 0) (attention_logit_heatmaps.png)

Case 4: The capital of France is not Berlin. Therefore, the capital of France is

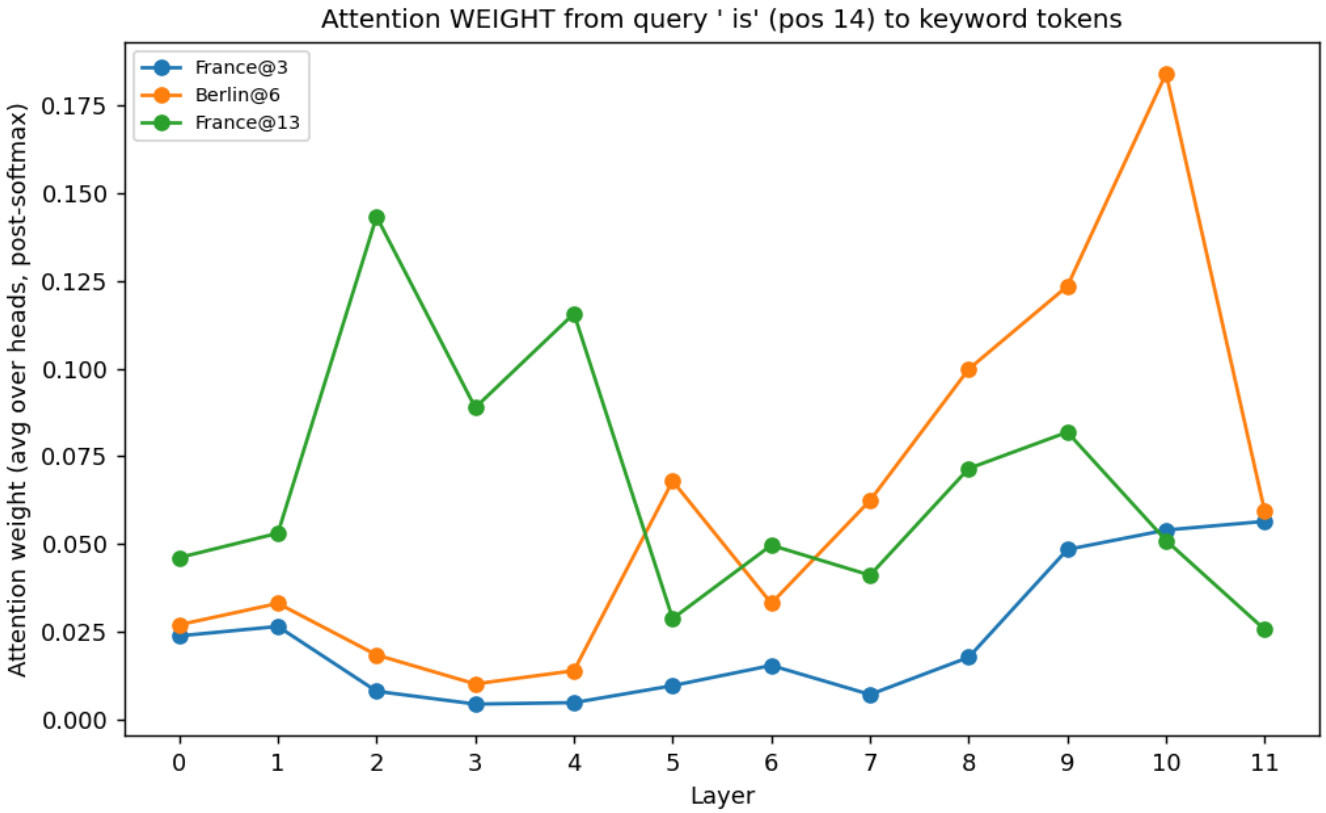
Generated: *not Berlin.* | different template, France/Berlin



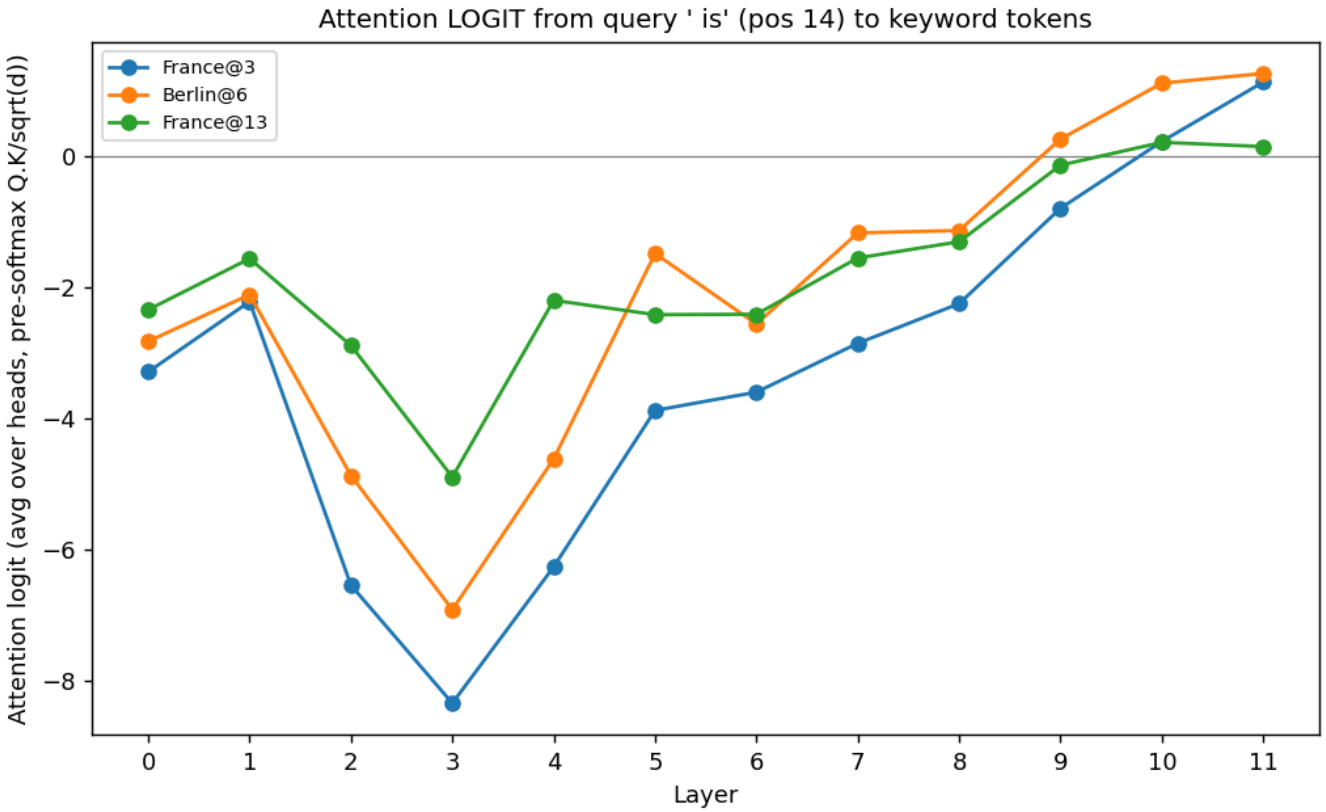
Case 4 -- Candidate probability (output layer) (candidate_probabilities.png)



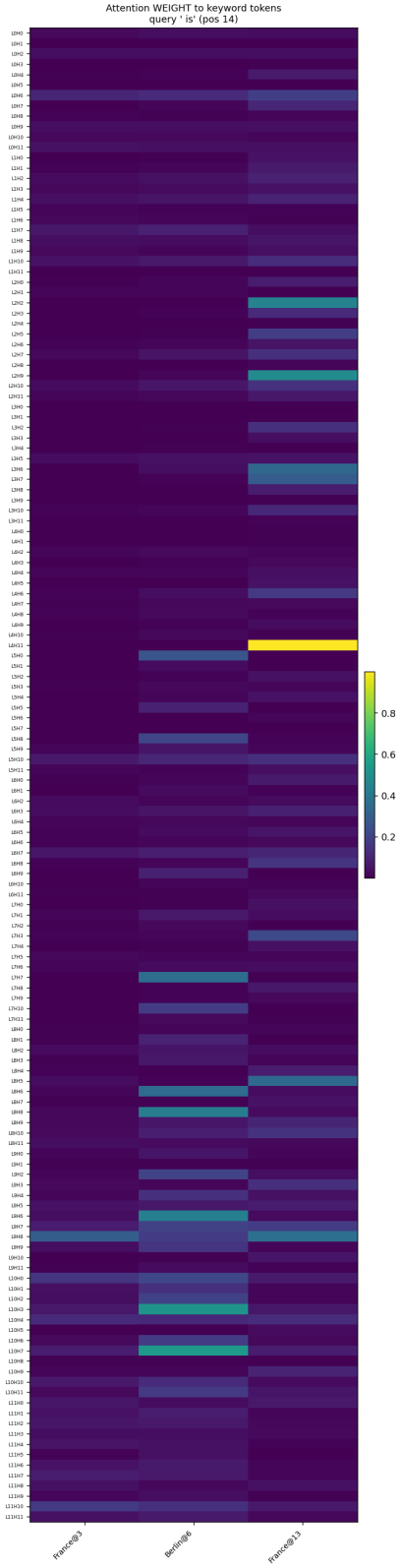
Case 4 -- Output-layer probability across full sequence (output_layer_heatmap.png)



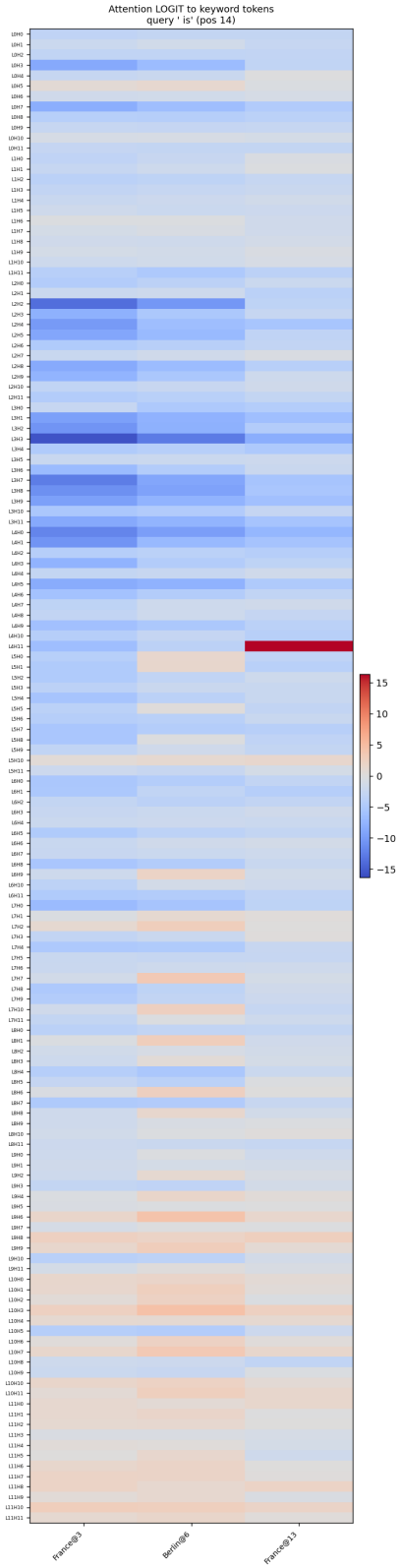
Case 4 -- Keyword attention WEIGHT by layer (keyword_attention_by_layer.png)



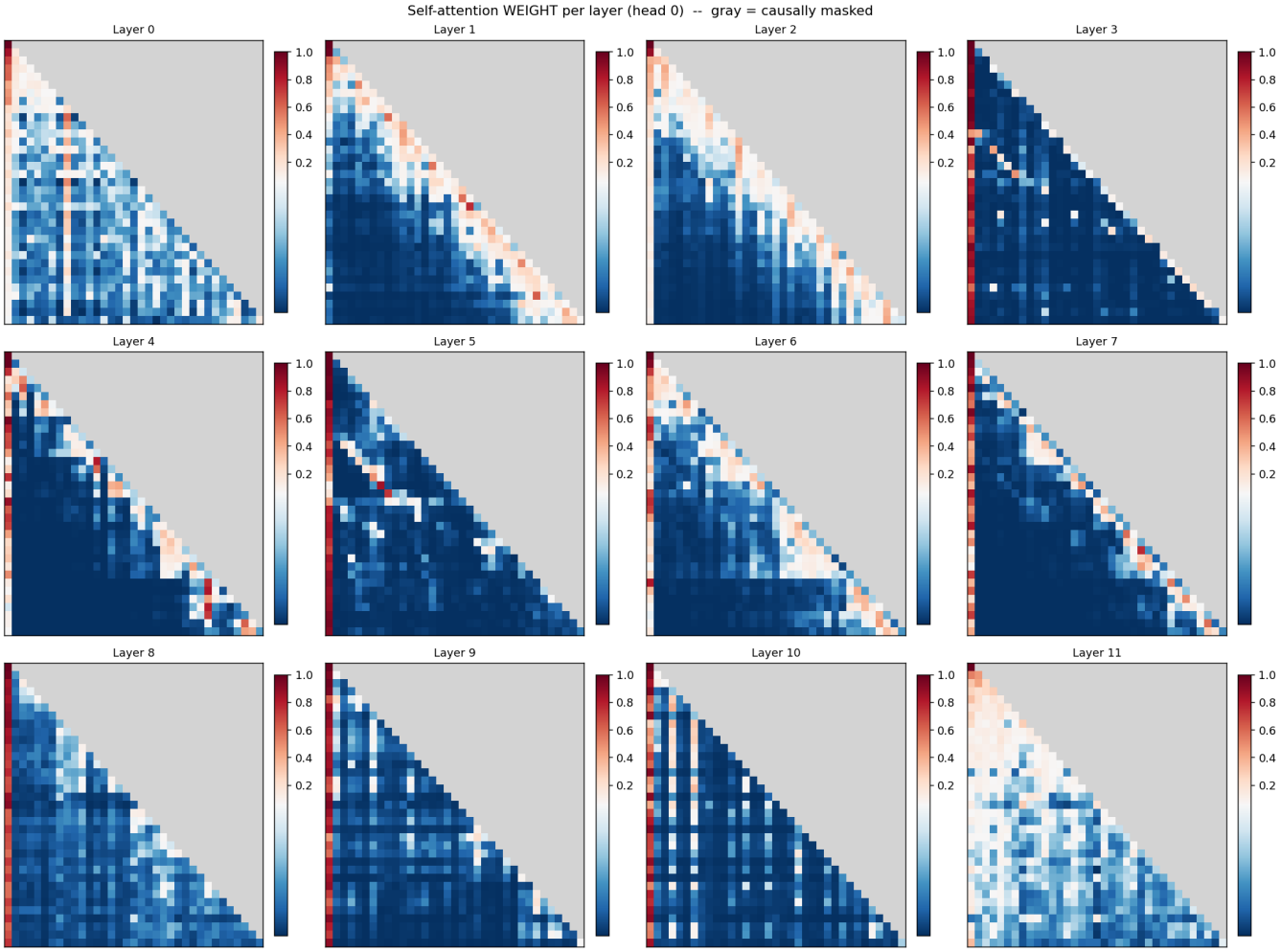
Case 4 -- Keyword attention LOGIT by layer (keyword_attention_logits_by_layer.png)



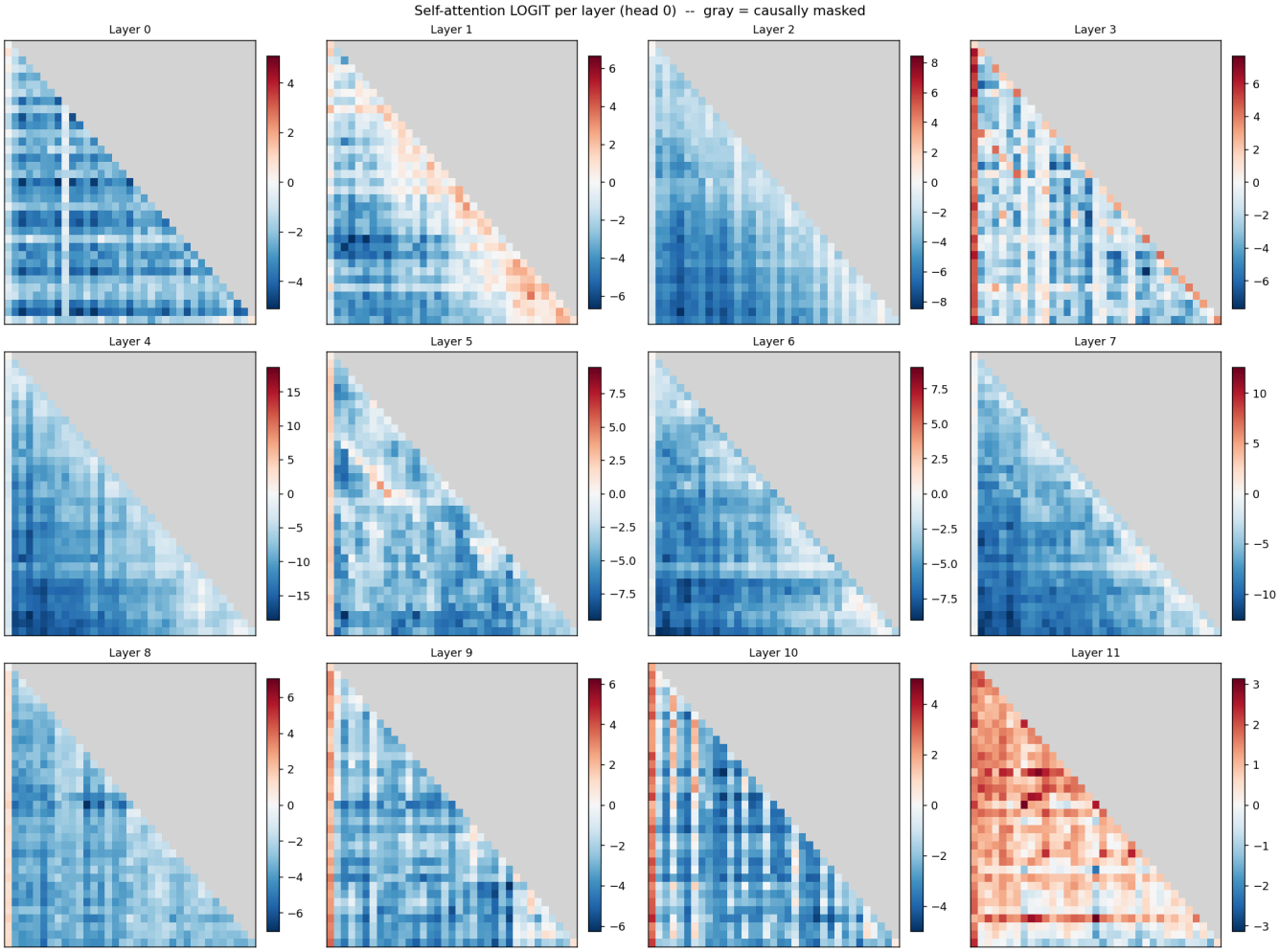
Case 4 -- Keyword attention WEIGHT by (layer, head) (keyword_attention_heads.png)



Case 4 -- Keyword attention LOGIT by (layer, head) (keyword_attention_logits_heads.png)



Case 4 -- Full per-layer self-attention WEIGHT grid (head 0) (attention_heatmaps.png)



Case 4 -- Full per-layer self-attention LOGIT grid (head 0) (attention_logit_heatmaps.png)